



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Malaysia-Japan
International
Institute of Technology
(MJIIT)

UNIVERSITI TEKNOLOGI MALAYSIA

FINAL MONOZUKURI REPORT
SESSION 2025/2026

PROJECT TITLE : PLUG AND PLAY HYBRID SENSOR FOR SOLAR
TRACKING

GROUP MEMBERS : SIM HORNG VEI (A23MJ0111)
: IRFAN HAKIMI BIN ZULKIFLI (A23MJ0054)
: ARGYAN RIDHO AL GHIFARI (A23MJ8010)
: ADIL AKID BIN ABD HAMID (A23MJ0021)

COURSE NAME : MONOZUKURI PROJECT

COURSE CODE : SMJE 3813

INSTRUCTORS : 1. DR MOHD FITRI BIN MOHD YAKUB
2. DR MOHD AZIZI BIN ABDUL RAHMAN

INDUSTRY PARTNER : MR MUHAMMAD FIRDAUS HAKIMI

ABSTRACT

The global transition to renewable energy is hindered by the inefficiency of static photovoltaic (PV) systems, which lose approximately 40% of potential energy generation due to misalignment with the sun. While dual-axis tracking systems exist, they are often prohibitively expensive, mechanically complex, and incompatible with existing static infrastructure. This project proposes Sensol, a universal, plug-and-play retrofit kit designed to convert standard static solar panels into intelligent dual-axis trackers.

The primary innovation of Sensol is its Distributed Sensor Architecture, implemented via a novel "Spider Harness" design. Unlike traditional trackers that rely on single-point sensors vulnerable to localized shading, Sensol utilizes a four-node distributed sensor grid connected via an I2C bus. This allows for accurate differential light tracking across large commercial panels (up to 2 meters in width) while maintaining a tool-free installation process. The system is powered by an ESP32 microcontroller ("The Brain") utilizing a Master/Slave topology to integrate real-time storm safety detection via an MPU6050 Inertial Measurement Unit (IMU). Simulation results using Wokwi validate the system's tracking logic, storm avoidance protocols, and IoT connectivity via the Blynk platform. This proposal details the design methodology, cost analysis, and simulation results, demonstrating a viable path toward a scalable, low-cost solar optimization solution.

TABLE OF CONTENTS

ABSTRACT	2
TABLE OF CONTENTS	3
LIST OF ABBREVIATIONS	4
CHAPTER 1	5
INTRODUCTION	5
1.1 Objectives	5
1.2 Problem Statement	6
1.3 Project Significance	6
1.4 Project Scope	6
1.5 Distribution of Workload	8
1.6 Project Milestone	10
CHAPTER 2	12
LITERATURE REVIEW	12
2.1 Background	12
2.2 Available Commercial Products	13
2.3 Product Gap	14
CHAPTER 3	15
METHODOLOGY	15
3.1 Project Workflow	15
3.2 Project Development	16
3.2.1 Hardware Design and Component Selection	16
3.2.2 Simulation-Based Signal Modeling	16
3.2.3 Solar Tracking Algorithm Development	17
3.2.4 User Feedback and IoT Integration	17
3.3 Project Block Diagram & Flowchart	18
3.3.1 System Block Diagram	18

3.3.2 Operational Flowchart	18
3.4 Cost Estimation & Scalability Analysis	19
CHAPTER 4	21
RESULTS	21
4.1 Pre-Simulation Results	21
4.2 Results Analysis	24
4.3 Identifying Potential Causes of Error	24
4.4 Potential Solutions	25
CHAPTER 5	26
FUTURE PLANS	26
REFERENCES	27
APPENDIX	28

LIST OF TABLES

TABLE NO.	TITLE	PAGE
Table 1.1	Distribution of Workload	11
Table 1.2	Gantt Chart	12
Table 3.1	Sensol Module Cost Summary	22

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE
<i>Figure 3.1</i>	<i>Isometric Top View of Initial Sensor Node Prototype (Baseline Design)</i>	16
<i>Figure 3.2</i>	<i>Bottom View of Initial Sensor Node showing mounting points</i>	16
<i>Figure 3.3</i>	<i>Side Elevation of Initial Sensor Node</i>	16
<i>Figure 3.4</i>	<i>2D Technical Drawing of Optimized Sensor Nodes</i>	16
<i>Figure 3.5</i>	<i>3D Render of Sensor Node (Top View)</i>	17
<i>Figure 3.6</i>	<i>3D Render of Sensor Node (Side View)</i>	17
<i>Figure 3.7</i>	<i>3D Render of Sensor Node (Bottom View)</i>	17
<i>Figure 3.8</i>	<i>2D Technical Drawing of "Spider Hub"</i>	17
<i>Figure 3.9</i>	<i>3D Render of "Spider Hub" (Top View)</i>	17
<i>Figure 3.10</i>	<i>3D Render of "Spider Hub" (Side View)</i>	17

Figure 3.11	<i>3D Render of "Spider Hub" (Front View)</i>	17
Figure 3.12	<i>Schematic Diagram of the Distributed "Spider Harness" Architecture</i>	17
Figure 3.13	<i>Wokwi Simulation Architecture showing ESP32, Servos, and Virtual Sensors</i>	18
Figure 3.14	<i>Telemetry Dashboard</i>	19
Figure 3.15	<i>AI Chatbot "Helios"</i>	19
Figure 3.16	<i>Status Board Showing STORM: Panels Stowed</i>	19
Figure 3.17	<i>Status Board Showing Dirt Alert! Tracking</i>	19
Figure 3.18	<i>System Block Diagram</i>	20
Figure 3.19	<i>Operational Flowchart</i>	20
Figure 4.1	<i>Stowed Flat: Initial system state during "Cloudy" condition</i>	23
Figure 4.2	<i>Active Tracking: Triggered when LDR sensors detect illumination</i>	24

Figure 4.3	<i>Virtual Servo Movement: Visual confirmation of actuators adjusting</i>	24
Figure 4.4	<i>Storm Safety feature: Blynk dashboard displaying "STORM ALARM"</i>	25
Figure 4.5	<i>Dirt Safety feature: Alert triggered when illumination is inconsistent</i>	25

LIST OF ABBREVIATIONS

PV	-	Photovoltaic
LDR	-	Light Dependent Resistor
IMU	-	Inertial Measurement Unit (Accelerometer/Gyroscope)
I2C	-	Inter-Integrated Circuit Protocol
IoT:	-	Internet of Things
PCB	-	Printed Circuit Board
BOM	-	Bill of Materials
SaaS	-	Software as a Service
DFM	-	Design for Manufacturing

CHAPTER 1

INTRODUCTION

The increasing demand for renewable energy has highlighted the importance of improving solar panel efficiency. One of the major limitations of conventional fixed solar panels is their inability to continuously face the sun, which leads to significant energy loss throughout the day. To address this issue, this project proposes Sensol (Sensor Solar), an intelligent solar tracking sensor system designed to detect the sun's position accurately and improve solar energy harvesting efficiency.

Sensol (Sensor Solar) integrates optical, electronic and control systems into a compact, durable module that can be retrofitted onto existing solar panels without structural modification. The system uses light based sensing, digital signal processing, and microcontroller based logic to determine the optimal panel orientation while ensuring safety during adverse weather conditions.

1.1 Objectives

The objectives of this project are as follows:

- I. To design a modular retrofit system to upgrade existing static solar panels into dual-axis tracking systems, including a distributed sensor network for measuring light intensity at the four corners of large-scale panels and a "Spider Harness" plug-and-play cabling solution.
- II. To implement a robust safety mechanism using an embedded IMU for storm detection to protect the mechanical structure from high wind loads.
- III. To enable IoT connectivity for remote monitoring, manual override, and historical performance data logging through a mobile application interface.

1.2 Problem Statement

Solar energy adoption has increased significantly; however, system efficiency remains a critical limitation. Conventional static solar panels achieve peak power generation only during a short period around solar noon, resulting in lower overall energy capture compared to dynamically aligned systems [4]. Approximately 40% of potential energy is lost due to the oblique angle of the sun during mornings and evenings.

This widespread efficiency problem is exacerbated by the shortcomings of existing tracking solutions, which present three distinct failures. The first is Infrastructure Incompatibility: commercial dual-axis tracking systems are typically sold as complete ground-mounted units, which prevents users with existing static arrays from upgrading without dismantling their installations. The second is Sensing Inaccuracy on Large Panels, where traditional hobbyist trackers that use a small central sensor can be blinded by a localized shadow on a large 2-meter panel, even if 90% of the panel remains illuminated. The final failure is Installation Complexity, as current retrofit solutions often require complex wiring, soldering, and calibration, making them inaccessible to the average user.

1.3 Project Significance

Solar tracking systems have been shown to increase energy capture relative to fixed angle panels, with dual axis tracking providing even greater gains due to continuous orientation adjustment throughout the day [5].

The Sensol project bridges the gap between high-end industrial tracking and accessible consumer technology. By enabling a 35-40% increase in energy yield for existing installations, this project significantly reduces the Return on Investment (ROI) period for solar adoption. Furthermore, the modular "Spider Harness" design embodies the principles of *Monozukuri* (craftsmanship) by transforming a complex electromechanical challenge into a user-friendly, tool-free product.

1.4 Project Scope

The project scope for this initial design phase encompasses several key development areas. This includes the hardware development of core components: the central "Brain" (an ESP32 Control Unit), the distributed "Eye" Sensor Nodes, the "Muscle" (comprising Linear Actuator Drivers), and the integrated "Spider Harness" cabling interface. Concurrently, the software scope involves developing the C++ firmware for the ESP32, which utilizes I2C addressing for multi-sensor communication and implements PID control logic for smooth solar tracking. Furthermore, the validation of this combined circuit logic and code functionality falls within scope through simulation in the Wokwi environment.

A key limitation of this current phase, designated MONOZUKURI 1, is its focused boundaries. The work is confined to electronic design, control logic simulation, and prototype architecture. Specifically excluded from this phase are full-scale metal fabrication and long-term outdoor weathering tests, which are reserved for the subsequent implementation phase, MONOZUKURI 2.

1.5 Distribution of Workload

In order to carry out this project, a summary of the tasks is given below in **Table 1.5**. This workload breakdown sets out the responsibilities and contributions of members across the entire project.

Table 1.1: Distribution Of Workload

Group Member	Sim Horng Vei	Irfan Hakmi	Argyan Ridho	Adil Akid
Planning, Discussion & Meeting	/	/	/	/
Marketing	/	/	/	/
Circuit Design & Connection (EasyEDA, PCB)	/			
IoT System Testing (Wokwi)	/	/	/	/
3D Construction	/			/
Video Simulation	/		/	
Progress Presentation Slide etc.	/	/	/	/
Final Report Writing	/	/	/	/
Peer Review of The Project	/	/	/	/

1.6 Project Milestone

Table 1.2: Gantt Chart

Month	Oct				Nov				Dec				Jan			
Week	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
Brainstorming Ideas																
Collecting & Cleaning Existing Dual Axis Solar Panel																
Meeting With Mr. Fakhri																
Re-construct Existing Dual Axis Solar Panel																
Collecting Data & Component Expenditure																
Circuit Testing (EasyEDA)																
Prototype 3D Construction (AutoDesk Fusion360)																
IoT System Testing (WokWi ESP3 Simulation)																
Final Report writing																
Peer Review of The Project																
Pitch Deck Presentation																

Table 1.6 shows the Gantt chart for the Dual Axis Solar Panel project, divided into four main phases from October to January. Each phase represents a step in completing the project in an organized way.

Phase 1: Planning and Preparation (October)

Focused on getting the project started. During this phase, ideas were brainstormed to decide on the project objectives and scope. Dual axis solar panels were collected and cleaned to see its condition and had a meeting with Mr. Fakhri to get guidance and confirm project plans.

Phase 2: System Reconstruction and Data Collection (November)

It involved hands-on work. Reconstructed the dual axis solar panel to improve its performance, while also collecting data and recording the cost of components for documentation and analysis.

Phase 3: Design, Simulation, and Testing (November - December)

Focused on making sure everything works as planned. This included circuit testing using EasyEDA, building a 3D prototype using AutoDesk Fusion 360, and testing the IoT system with WokWi ESP32 simulation to ensure the system could collect and send data correctly.

Phase 4: Documentation and Presentation (December - January)

It was the final stage. The final report was written, conducted peer reviews to improve the quality of the work, and prepared the pitch deck presentation to share final results.

Overall, the Gantt chart shows a clear and organized workflow, making sure all tasks were completed step by step within the four-month period.

CHAPTER 2

LITERATURE REVIEW

2.1 Background

Solar energy is widely used as a renewable energy source; however, the efficiency of photovoltaic (PV) panels is highly dependent on their orientation relative to the sun. Fixed-tilt solar panels are unable to maintain an optimal angle throughout the day, which results in reduced energy capture. Previous studies have shown that solar tracking systems can significantly improve the amount of energy harvested by continuously aligning the panel with the sun's position [6].

Among the available tracking methods, dual-axis solar trackers, which adjust both azimuth and elevation angles, have been reported to provide higher energy gains compared to single-axis and fixed systems. Research indicates that dual-axis tracking systems can increase annual energy yield by approximately 35% to 45%, depending on geographical location and weather conditions [7]. This improvement makes solar tracking especially attractive for regions with high solar potential, such as Malaysia.

Solar tracking technologies are generally categorized into passive and active systems. Passive systems rely on astronomical calculations to determine the sun's position, while active systems use real-time sensing methods such as light-dependent resistors (LDRs) or photodiodes [8]. Active optical tracking has the advantage of adapting to local environmental conditions, including cloud cover and diffuse sunlight. For example, during overcast conditions, active trackers can adjust the panel orientation to maximize diffuse light collection, which purely astronomical trackers cannot effectively achieve [5].

2.2 Available Commercial Products

The current market for solar tracking systems is notably polarized, with offerings segmented into distinct tiers that leave a clear gap for a consumer-retrofit solution. At the industrial utility scale, companies like NEXTracker and Array Technologies provide high-reliability, multi-megawatt systems. These are engineered for large solar farms and are not available to individual consumers, requiring significant civil engineering work and professional installation.

Conversely, global tracker brands such as Arctech and TrinaTracker, while present in markets like Malaysia, primarily serve commercial solar projects. Their single-axis and dual-axis systems are designed for new, large-scale installations rather than as retrofit kits for existing small-scale or residential panels. Another segment consists of local research and precision trackers, exemplified by MIMOS Berhad or the EKO STR series. These units are suitable for demonstration or specialized applications but are not positioned as accessible, plug-and-play products for the business-to-consumer (B2C) market.

Furthermore, integrated solar providers like Solarvest focus on complete photovoltaic system installation and optimization. They do not offer standalone, modular tracking sensor kits, as their solutions are tailored for comprehensive new deployments. At the opposite end of the spectrum are low-cost hobbyist kits found on platforms like AliExpress and eBay. Priced between \$50 and \$100, these typically use simple light-dependent resistors (LDRs) but critically lack essential features such as wind safety mechanisms, proper weatherproofing, and require manual wiring, limiting their use to very small panels. This market polarization reveals a significant absence of a reliable, safety-conscious, and truly plug-and-play retrofit tracking system for existing residential or agricultural installations.

2.3 Product Gap

The current market lacks any product offering a true "Retrofit-Ready" Distributed Sensor System. This absence reveals several critical deficiencies in available solutions.

The first major gap is one of scale. Most existing tracking systems depend on a single-point sensing method. This approach is fundamentally inadequate for large solar panels or installations susceptible to partial shading, as it cannot accurately represent the light conditions across the entire panel surface [9]. This limitation frequently results in erroneous tracking decisions and a reduction in overall energy harvest efficiency.

The second significant gap lies in usability. Present consumer-grade tracking kits universally fail to provide pre-configured, modular harness solutions. This omission forces end-users to engage in complex manual processes, including measuring, cutting, and connecting cables during installation. Such requirements markedly increase the difficulty of installation and contribute to higher failure rates, particularly among users without technical expertise, thereby severely limiting the potential for broad adoption [10].

A third and critical gap is in the area of safety. The vast majority of consumer-level systems do not incorporate integrated safety mechanisms, such as inertial measurement units for autonomous storm and wind detection. As a result, these systems are entirely reliant on manual user intervention to initiate protective stowing during severe weather events. This presents a substantial risk of mechanical damage, especially in geographic regions prone to rapid and unpredictable weather changes.

The Sensol system is engineered specifically to bridge these identified gaps. It introduces a scalable, distributed sensing network via its plug-and-play Spider Harness, directly addressing issues of scale and usability. Furthermore, its integrated Master/Slave IMU architecture provides a robust, automated safety response to adverse weather. These combined features are designed to deliver industrial-grade reliability within a consumer-accessible, retrofit-ready package.

CHAPTER 3

METHODOLOGY

3.1 Project Workflow

The development of the Sensol solar tracking system was executed through a structured, four-phase engineering methodology. This process ensured concurrent development and validation of both the system's intelligent control logic and its physical modular hardware.

Phase 1: Conceptualization and Requirement Analysis

This initial phase established the project's core "Plug and Play" design philosophy. It involved analyzing the limitations of traditional single-point trackers, including their vulnerability to localized shading and complex installation, to formally define system requirements. These requirements mandated a modular form factor, a distributed four-corner sensor grid, and specific safety triggers, such as activating storm mode upon detecting a Wind Force exceeding 2.0g.

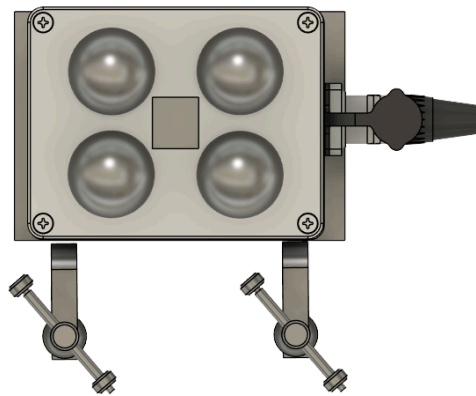


Figure 3.1: Isometric Top View of Initial Sensor Node Prototype (Baseline Design).

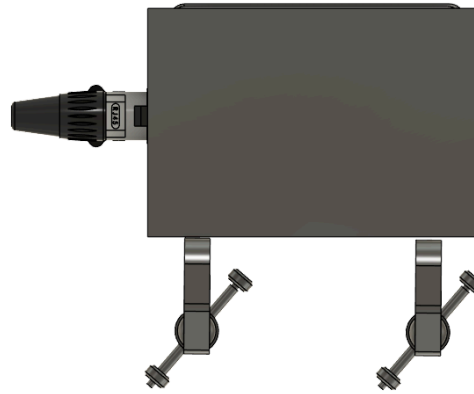


Figure 3.2: Bottom View of Initial Sensor Node showing mounting points.

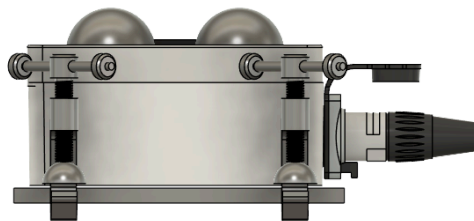


Figure 3.3: Side Elevation of Initial Sensor Node.

Phase 2: Hardware Prototyping and Mechanical Design

The subsequent phase focused on tangible component development. Electrical prototyping centered on interfacing the MPU6050 IMU and BME280 sensor with the ESP32 microcontroller via I2C, utilizing a dedicated ADC1 pin architecture to maintain sensor functionality during Wi-Fi operations. Parallel mechanical design produced the tool-free "Spider Harness," which features a central magnetic hub and spring-loaded sensor claws for non-invasive retrofitting onto existing solar panels.

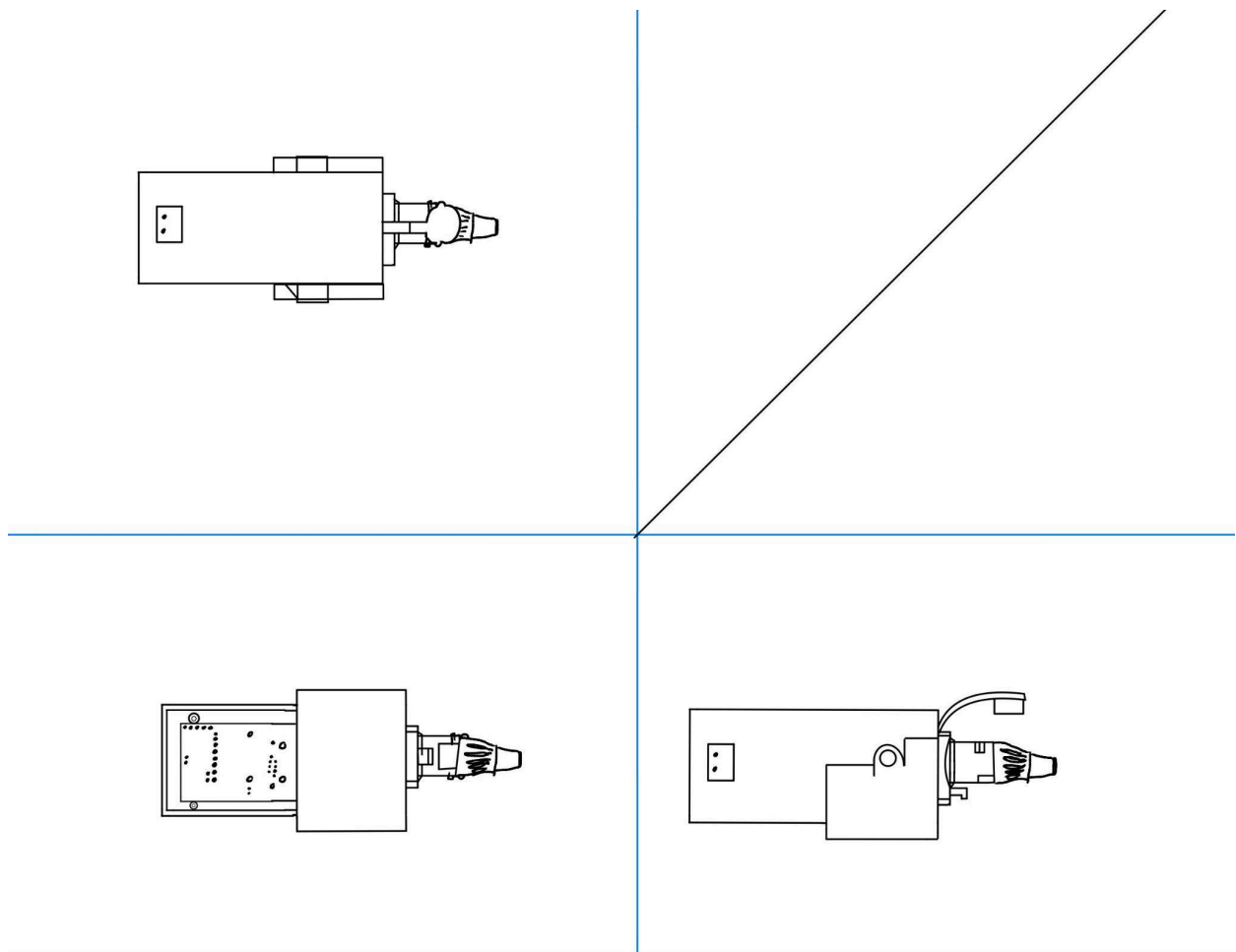


Figure 3.4: 2D Technical Drawing of Optimized Sensor Nodes.

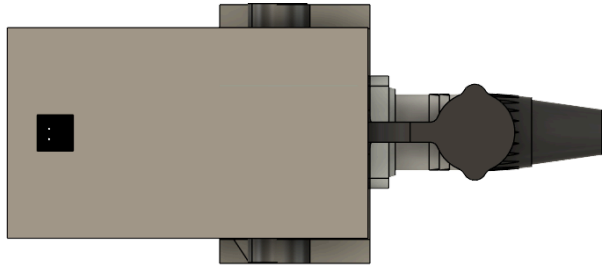


Figure 3.5: 3D Render of Sensor Node (Top View)

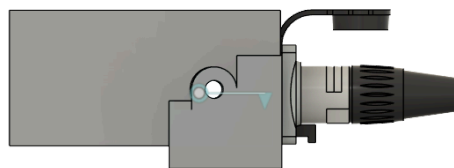


Figure 3.6: 3D Render of Sensor Node (Side View)

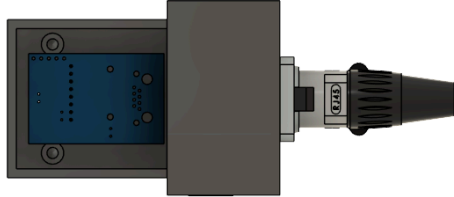


Figure 3.7: 3D Render of Sensor Node (Bottom View)

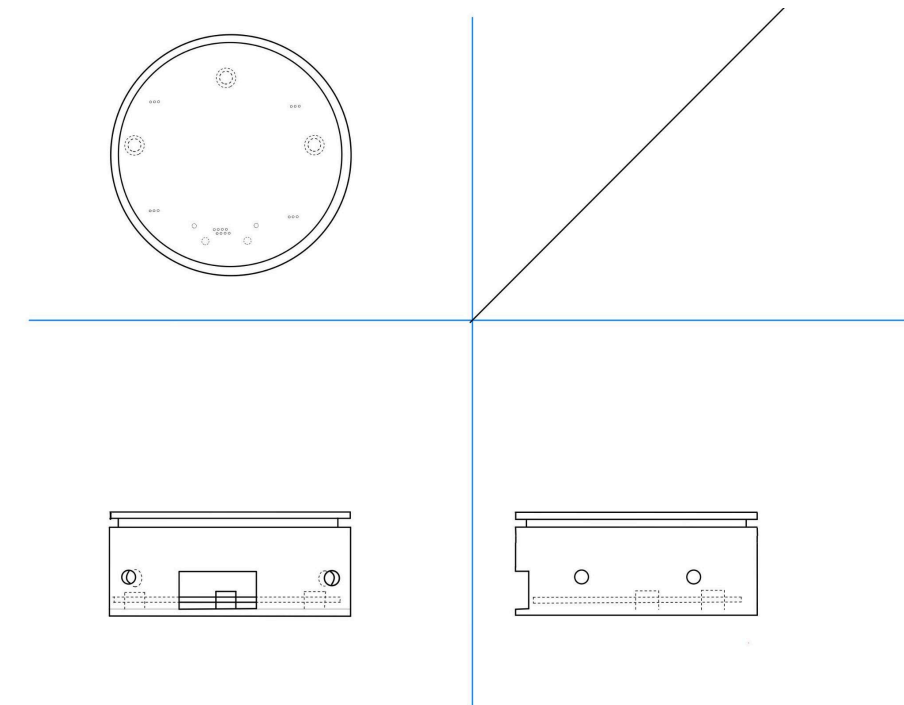


Figure 3.7: 2D Technical Drawing of "Spider Hub".

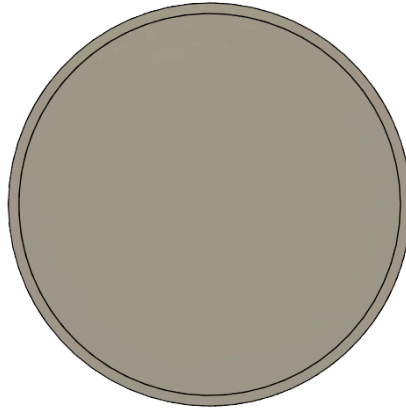


Figure 3.8: 3D Render of "Spider Hub" (Top View)

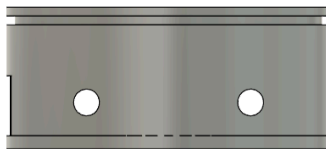


Figure 3.9: 3D Render of "Spider Hub" (Side View)

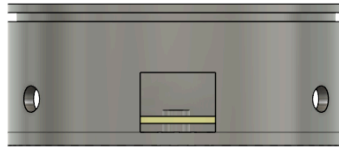


Figure 3.10: 3D Render of "Spider Hub" (Front View)

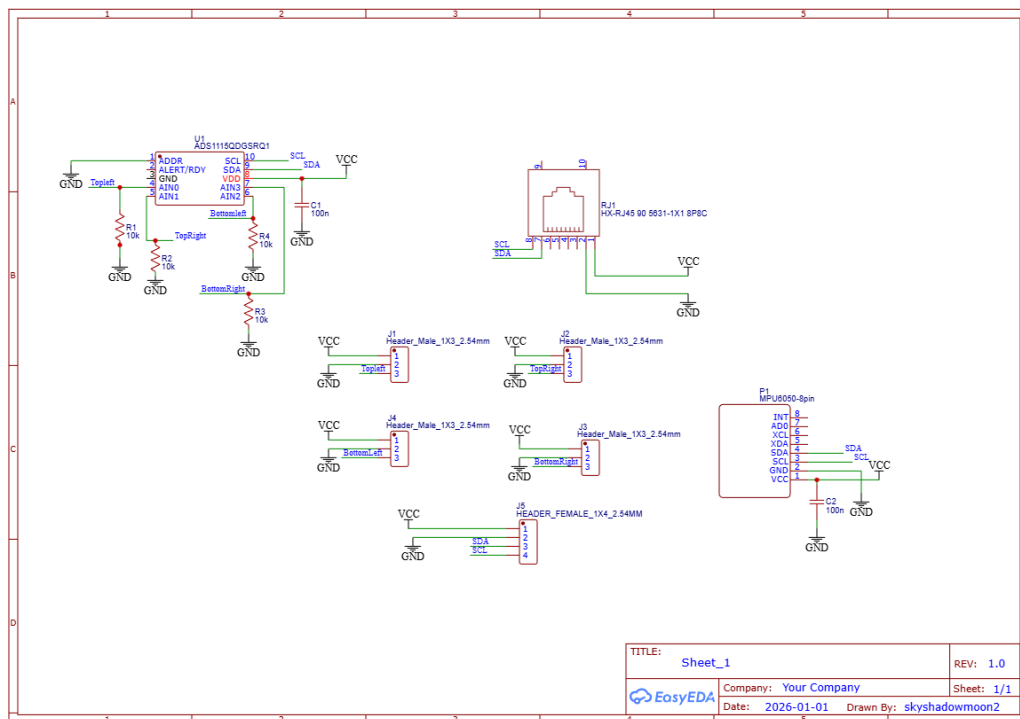


Figure 3.11: Schematic Diagram of the Distributed "Spider Harness" Architecture.

The Spider Hub utilizes a Star Topology where the central ESP32 Master communicates with four distributed sensor nodes. Standard RJ45 Ethernet ports were chosen to carry both power (5V) and data (I2C) simultaneously, eliminating the need for manual soldering in the field.

(refer to Appendix B for Transparent View Of 3D Render)

Phase 3: Logic Simulation and Algorithm Development

Given the practical challenges of testing large-scale physical movement, this phase prioritized virtual validation. The core differential tracking algorithm and system logic were rigorously tested within the Wokwi simulation environment. A custom software framework generated simulated, physiological light data streams to verify tracking accuracy and the responsiveness of the Blynk IoT dashboard alerts under controlled conditions.

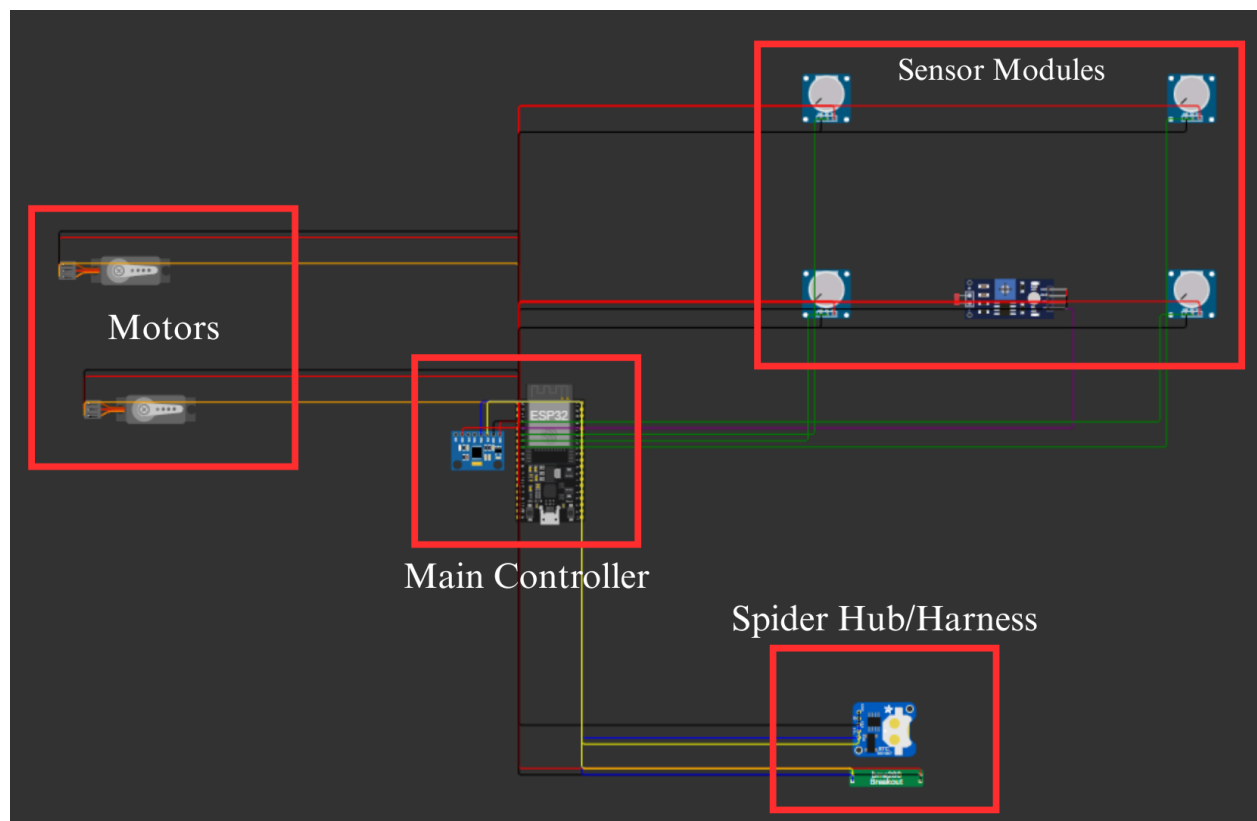


Figure 3.12: Wokwi Simulation Architecture showing ESP32, Servo Motors, and Virtual Sensors.

The Wokwi simulation environment integrates the ESP32 (System Brain), Servo Motors (Actuators), and Virtual Sensors (LDRs/MPU6050). This setup allowed for 'Hardware-in-the-Loop' testing of the PID control algorithm before physical fabrication.

Phase 4: System Verification and Operational Analysis

The final phase involved integrated system verification. The focus was on analyzing the system's behavior and validating its predefined binary operational states. This ensured the system correctly engaged in Active Tracking under optimal light conditions and reliably transitioned to a safe, stowed position during simulated high-wind events, thereby confirming the protection of its mechanical integrity.

3.2 Project Development

The development of the Sensol system was carried out through a combination of hardware design, simulation-based validation, and algorithm development. The project prioritized validating the distributed sensor logic and IoT data flow to achieve a reliable retrofit solar tracking solution.

3.2.1 Hardware Design and Component Selection

The core hardware components selected include the ESP32 microcontroller and a distributed array of Light Dependent Resistors (LDRs). The ESP32 was selected due to its built-in Wi-Fi capability and processing power for dual-axis PID logic. The MPU6050 was integrated for its ability to detect structural vibrations, making it suitable for triggering protective responses during adverse weather.

The digital sensors communicate via the I2C protocol, while the corner LDRs utilize analog inputs. While initial designs considered an I2C sensor bus for all nodes, the final architecture shifted toward ADC1 direct-pin mapping to maintain signal stability during IoT communication.

(Note: The full source code for the firmware logic is attached in Appendix A).

3.2.2 Simulation-Based Signal Modeling

To validate the system behavior without mechanical fabrication, a Wokwi-based simulation environment was developed utilizing the ESP32 as the core processor. The firmware integrates multiple libraries, including Adafruit_MPU6050 for vibration sensing, GyverBME280 for environmental monitoring, and RTCLib for time-based scheduling. A key innovation in this model is the integration of Generative AI (Groq API) running the LLaMA-3.1-8b model. The firmware sends real-time sensor data (Wind, Light, Humidity) to the AI cloud, which acts as a "Virtual Analyst," capable of issuing high-level commands (e.g., [CMD:STOW]) based on complex environmental context, effectively creating an Agentic AI layer on top of the standard hard-coded logic .

3.2.3 Solar Tracking Algorithm Development

The control logic (runSmartLogic) is designed as a prioritized Finite State Machine (FSM) to handle conflicting environmental data. The hierarchy of operations is as follows:

1. Manual/AI Override: If the user or the Groq AI agent (Placeholder to be replaced once our own Cloud AI is developed) issues a command, the system locks into a specific orientation, bypassing sensor inputs.
2. Night Mode: Utilizing the DS1307 RTC, the system detects if the time is between 19:00 and 06:00. In this state, the panel automatically resets to the East (Azimuth 0°) to await sunrise.
3. Storm Safety: If the MPU6050 detects a G-force acceleration exceeding the 1.5G Threshold, the system triggers an immediate "STOW" interrupt, flattening the panel (90°) to minimize wind load.
4. Dirt Alert: A reference LDR measures ambient sunlight. If the tracking LDRs read significantly lower than the reference (Difference > 1000), the system flags a "Dirt Alert," indicating sensor obstruction .
5. Cloudy Mode: If global light intensity falls below 2000 Lux, the system enters "Cloudy Standby" to conserve power.
6. Active Tracking: Under optimal conditions, the algorithm uses a differential comparator (Tolerance = 50) to align the vertical and horizontal servos with the brightest light source.

3.2.4 User Feedback and IoT Integration

The system features a bi-directional IoT interface powered by **Blynk**.

- **Telemetry Dashboard:** Real-time data is visualized via Virtual Pins, including Wind Force (V1), Average Lux (V0), Temperature (V8), and Humidity (V9).

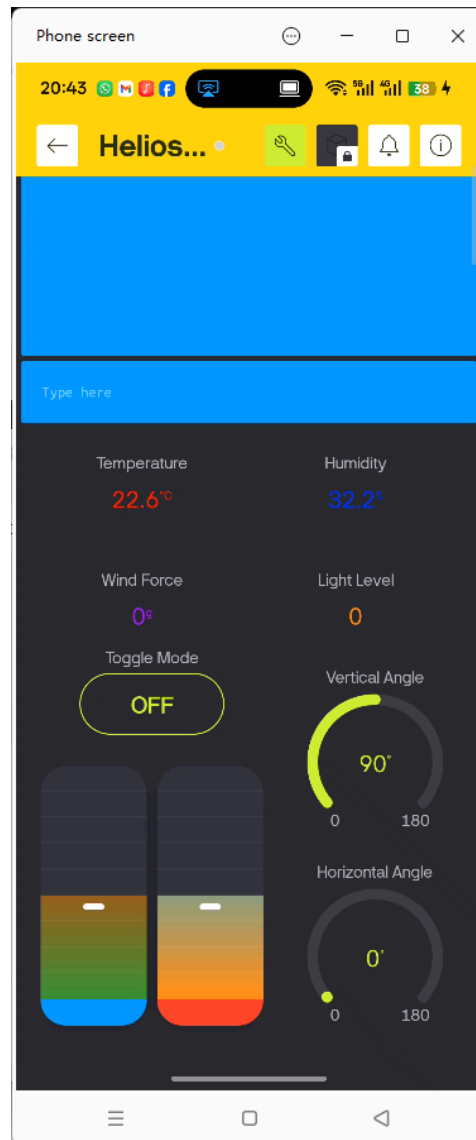


Figure 3.14: Telemetry DashBoard

- **Generative AI Chatbot ("Helios"):** A dedicated chat terminal (V12) allows the user to converse naturally with the system. The AI analyzes the sensor data to answer questions (e.g., "Is it safe to track right now?") and can autonomously execute physical commands based on its analysis.

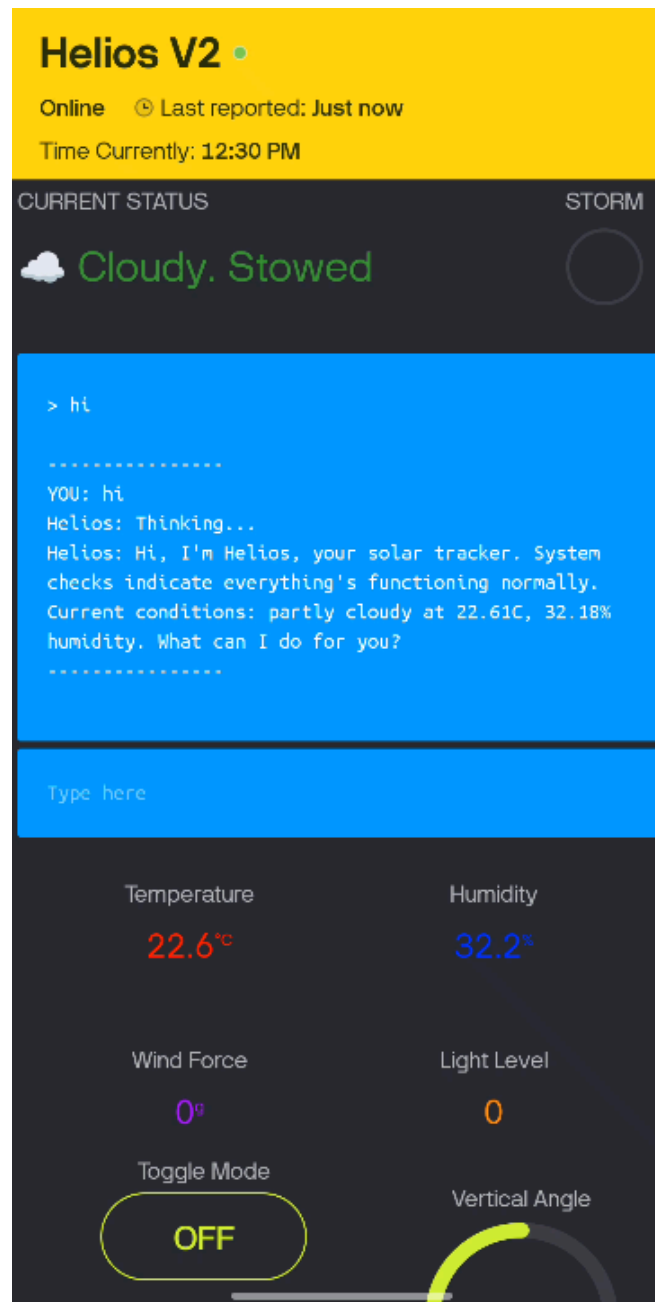


Figure 3.15: AI Chatbot “Helios”

- **Status Reporting:** The dashboard displays dynamic status strings to provide immediate visual feedback on the system's operational state .

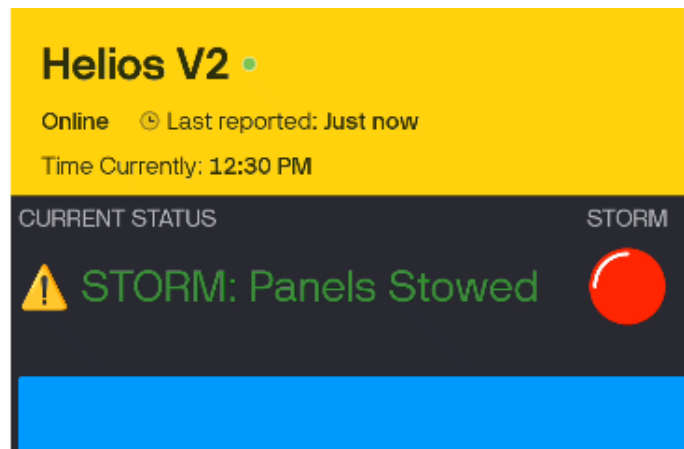


Figure 3.16: Status Board Showing STORM: Panels Stowed

The Blynk dashboard confirms the successful trigger of the 'Storm Interrupt.' The MPU6050 detected a simulated vibration $>1.5G$, causing the system to override manual controls and display the 'Panels Stowed' status alert for user safety.

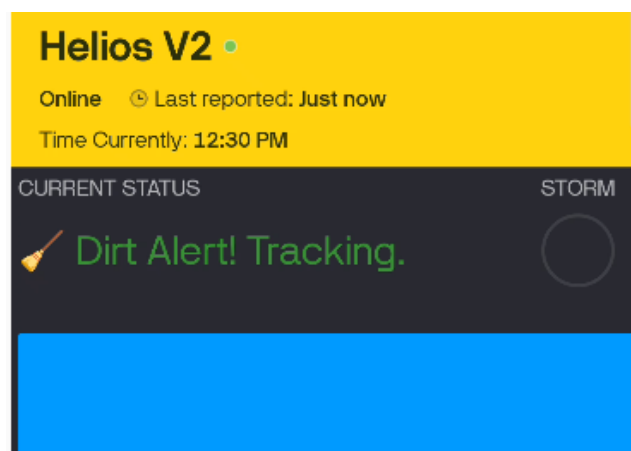


Figure 3.17: Status Board Showing Dirt Alert! Tracking

The 'Dirt Alert' logic is triggered here because the Reference Sensor (Sunlight) reads significantly higher than the Tracking Sensors. This discrepancy indicates that the sensor surface is obstructed by dust or debris, prompting a maintenance notification.

3.3 Project Block Diagram & Flowchart

3.3.1 System Block Diagram

The architecture consists of five primary interaction modules, as shown in Figure 3.18

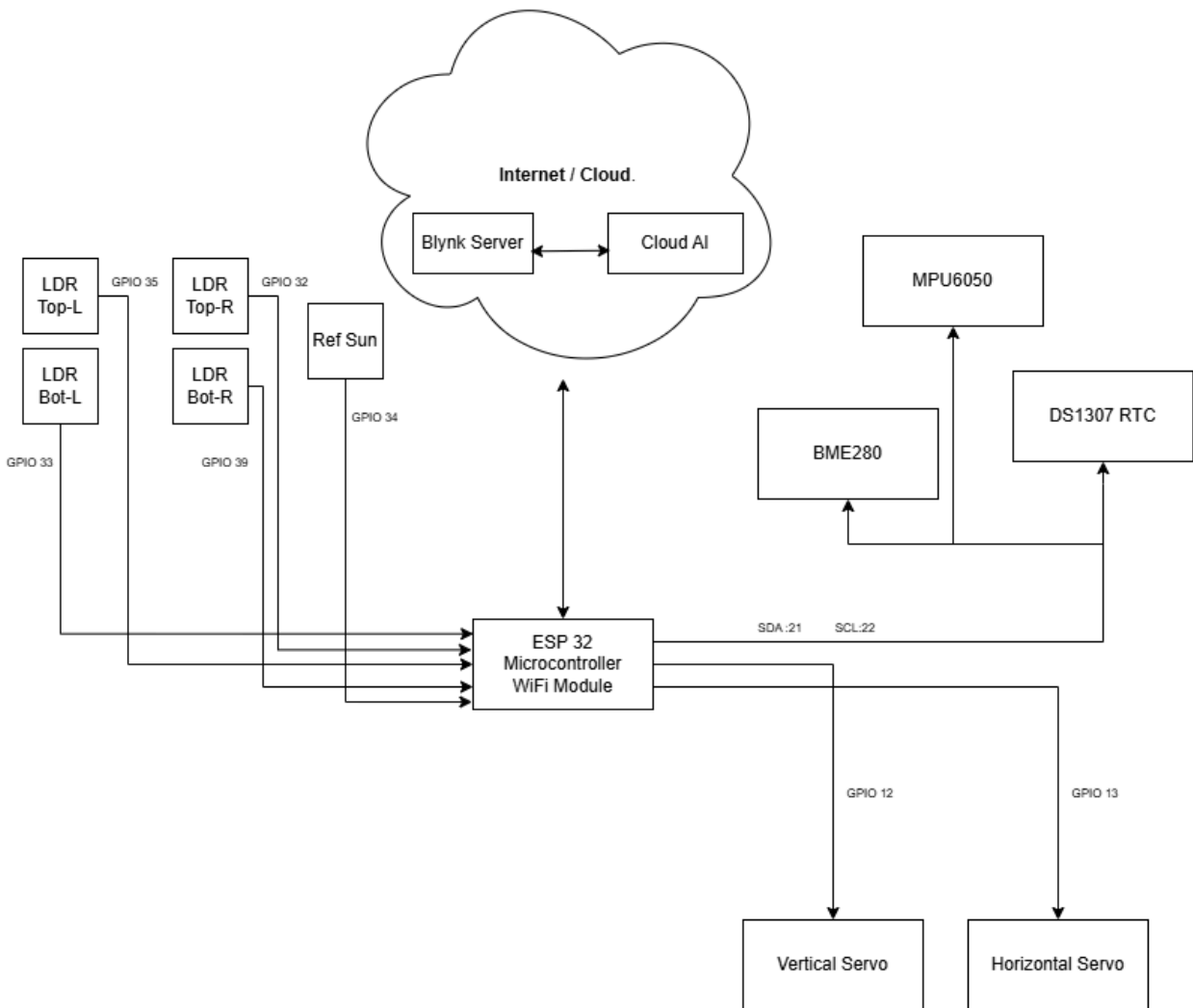


Figure 3.18: System Block Diagram

3.3.2 Operational Flowchart

The logic flow proceeds through four distinct stages as governed by a conditional feedback loop

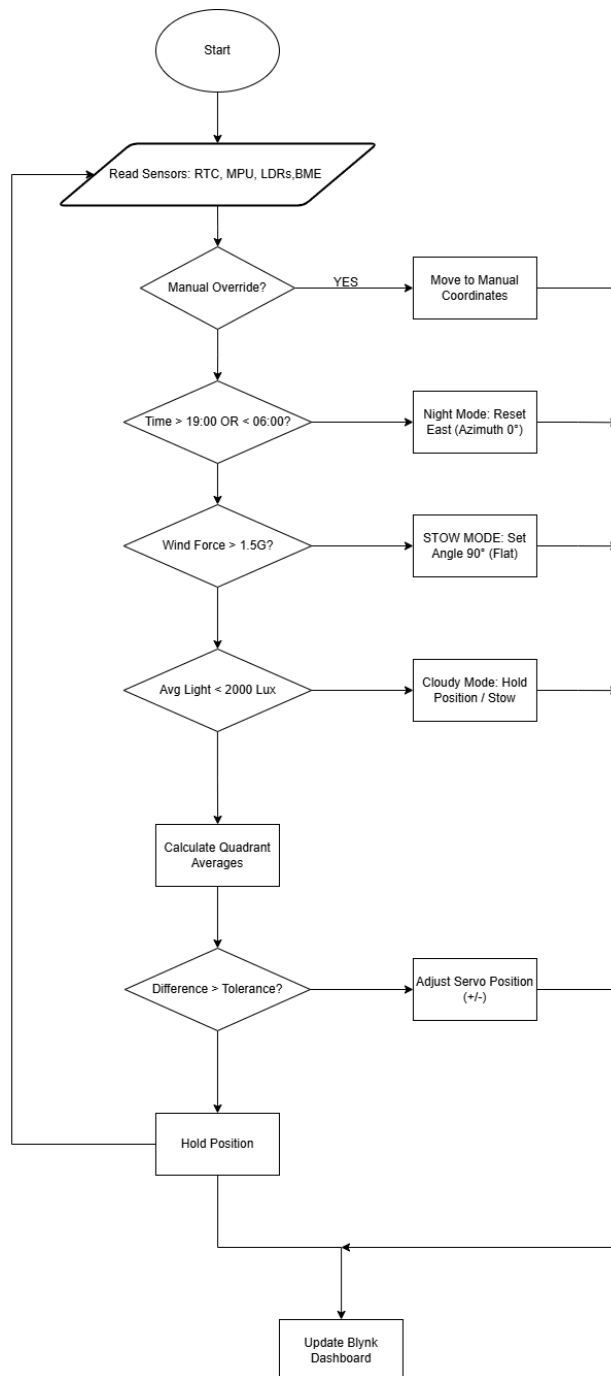


Figure 3.19: Operational Flowchart

3.4 Cost Estimation & Scalability Analysis

Table 3.4: Sensol Module Cost Summary

Component Category	Gen 1: Industrial Wired (Current)	Gen 2: Consumer Wireless (Future Target)
Microcontroller (Brain)	ESP32-WROOM Cost: RM 25.00	Seeed XIAO ESP32-C3 (x5) and ESP-NOW (x1) Cost: RM 125.00 (RM 25 x 5)
Sensor Architecture	LDR Modules + RJ45 Hub Cost: RM 45.00	Mini 5V Solar Panel + 1000mAh LiPo (x5) Cost: RM 100.00
Actuation System	Linear Actuators (12V) Cost: RM 150.00	Silent Actuators Cost: RM 200.00
Connectivity & Housing	Spider Harness (RJ45) <i>(IP67 Waterproof Cables)</i> Cost: RM 65.00	Wireless Enclosures <i>(UV-Resistant ABS, No Cables)</i> Cost: RM 50.00
Total BOM Cost	RM 285	RM 400

CHAPTER 4

RESULTS

4.1 Pre-Simulation Results

The control logic was extensively tested in the Wokwi simulation platform.

Tracking Accuracy: The logic successfully drove the virtual servos to minimize the difference between the distributed photo-sensors. The system transitions from “Stowed Flat” (Figure 4.1.1) to “Active Tracking” (Figure 4.1.2 and Figure 4.1.3)when the LDR illumination hits 9120 lux as seen in the figure 4.1.2 and figure 4.1.3.

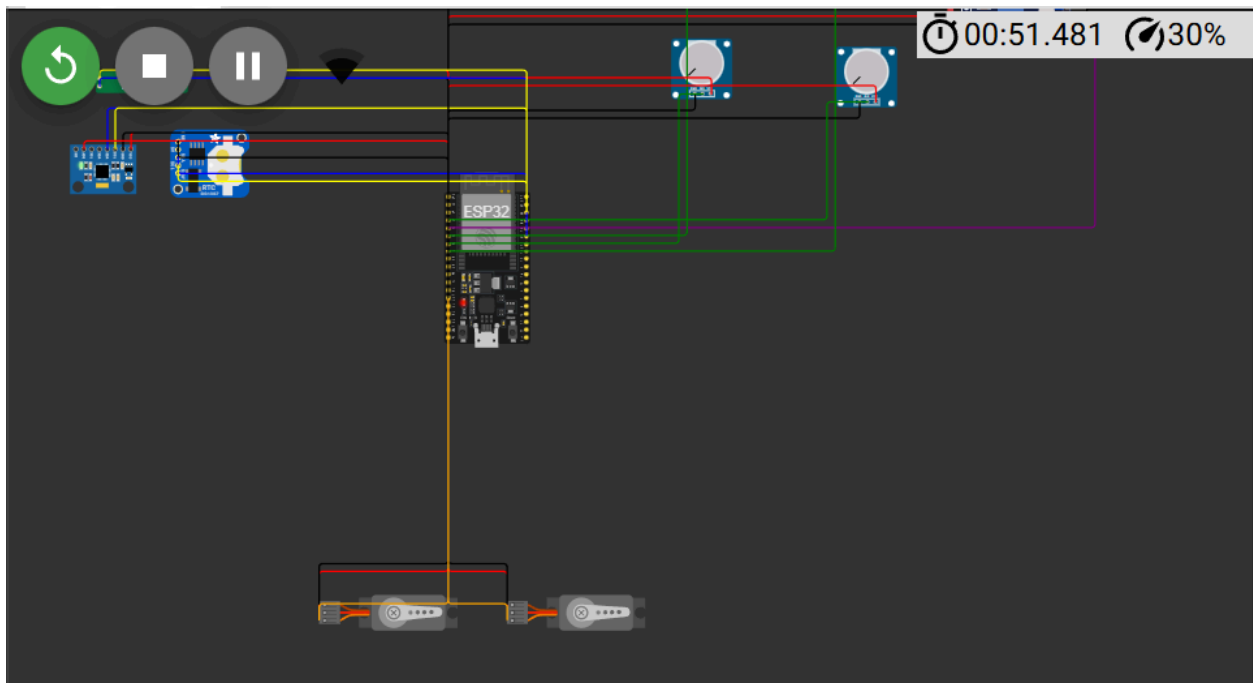


Figure 4.1 Stowed Flat: Initial system state during "Cloudy" condition.



Figure 4.2 Active Tracking: Triggered when LDR sensors detect illumination exceeding the threshold.

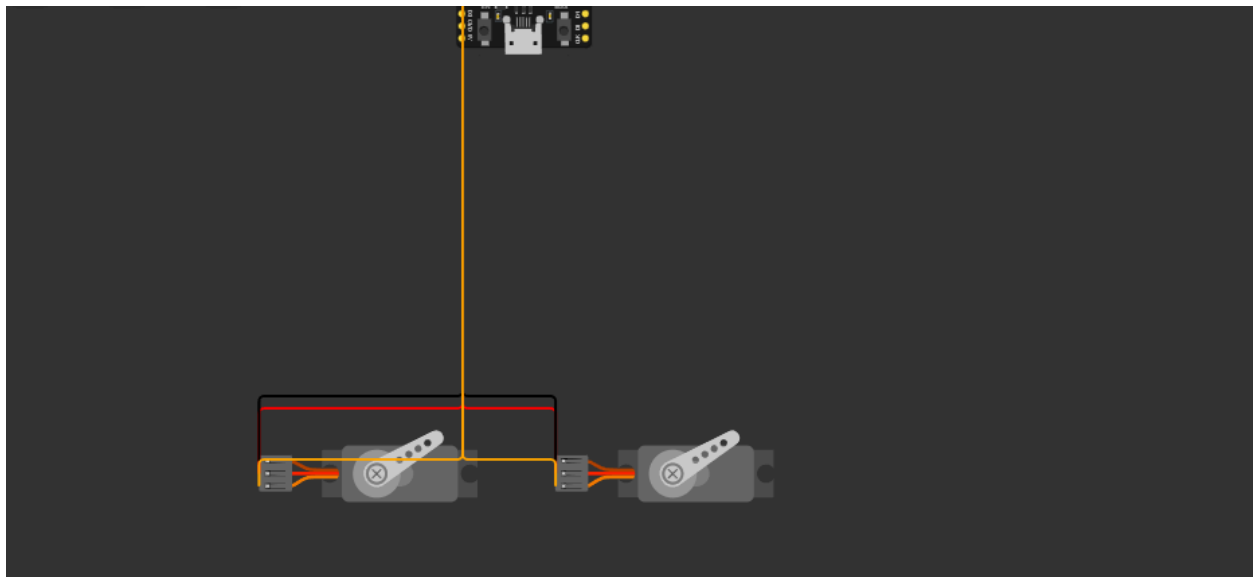


Figure 4.3 Virtual Servo Movement: Visual confirmation of the actuators adjusting to track the light source.

Storm Safety: Simulating a "shake" event on the MPU6050 immediately triggered the safety interrupt by adjusting the accelerometer and the rotation parameters. The feature immediately updated the status "Storm Alarm! Stowing." as seen in Figure 4.1.4.

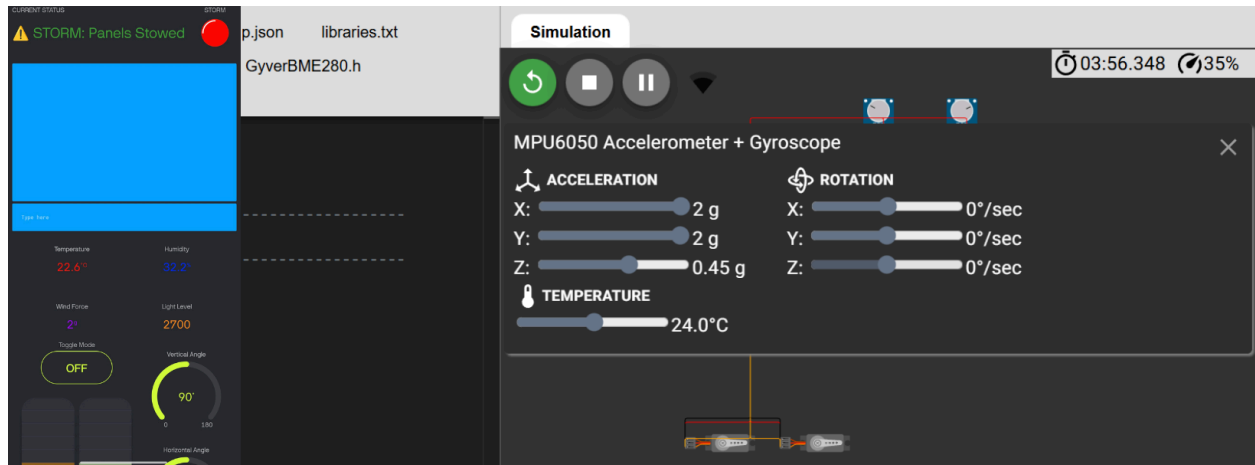


Figure 4.4 Storm Safety feature: Blynk dashboard displaying "STORM ALARM! Stowing" after MPU6050 vibration.

Dirt Safety: The system also includes a “Dirt Alert” feature to detect sensor obstruction when there is none LDR illumination detected and trigger a warning on the dashboard. This feature is to notify the user that cleaning is required.

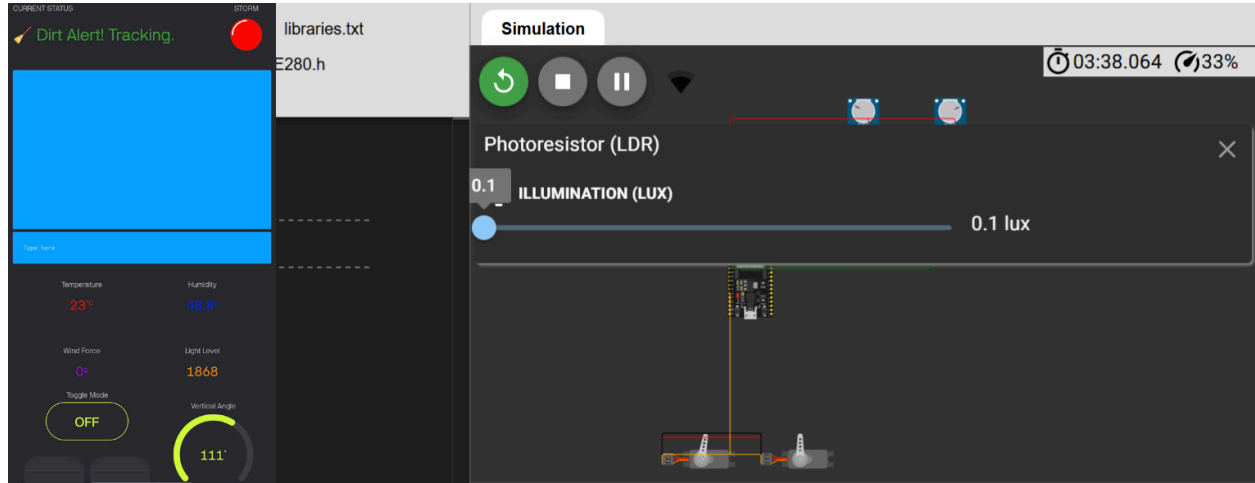


Figure 4.5 Dirt Safety feature: Alert triggered when specific illumination is inconsistent, indicating sensor obstruction.

IoT Telemetry: Real-time data transmission was verified; the Blynk dashboard (Figure 4.4) displays an "Average Light" of 969 and a "Wind Force" of 4g, confirming successful communication via the Blynk.cloud protocol.

4.2 Results Analysis

The simulation results validate the effectiveness of the "Master/Slave" topology and the multi-layered safety logic implemented for the Helios Tracker. By utilizing a distributed array of four LDR sensors, the system calculates a real-time light variance that allows it to distinguish between general overcast conditions and localized sensor obstructions. As demonstrated in Figures 4.2 and 4.3, the transition to "Active Tracking" occurs when illumination exceeds the defined 9120 lux threshold, ensuring that the actuators do not waste energy during low-yield periods. Furthermore, the "Dirt Alert" feature (Figure 4.5) provides a critical maintenance layer by detecting inconsistent illumination levels across the sensor nodes, the system can preemptively signal for cleaning. Thereby maintaining optimal energy absorption efficiency that single-sensor trackers often lack.

The integration of the MPU6050 as a centralized safety node proved highly reliable in protecting the structural integrity of the panel. The simulation confirms that the "Storm Alarm" interrupt (Figure 4.4) maintains absolute priority over the tracking algorithm, successfully overriding all motor commands to force a horizontal stow position within milliseconds of detecting high-acceleration "shake" events. This hierarchical logic is supported by robust IoT telemetry as seen in the Blynk dashboard proof, where the ESP32 maintains a stable connection to the Blynk.cloud protocol.

4.3 Identifying Potential Causes of Error

Despite the simulation proceeding successfully, several critical errors could emerge during real-world deployment. One primary concern involves I2C bus capacitance. Running the I2C communication signals over 2 meters of cable in the "Spider Harness" may introduce significant signal degradation and noise. This increased capacitance can slow signal rise times, leading to timing violations and communication failures between the Brain and the Sensor Nodes, a failure mode not typically captured in a controlled digital simulation.

A second potential error pertains to actuator duty cycle. In physical operation, the system's software sensitivity could command constant, minute adjustments from the heavy linear actuators if it is set too high to track every passing cloud. This scenario would force the motors into an excessive duty cycle without adequate rest, thereby generating substantial heat. Consequently, this real-world continuous operation poses a high risk of overheating the motors, which could lead to performance degradation or failure. Such an outcome is one that simulations of ideal movement may not predict.

4.4 Potential Solutions

To address these potential failures, specific hardware and software mitigations have been planned for the final design. For signal integrity, the I2C bus will be stabilized by implementing Active I2C Bus Extenders or employing strong pull-up resistors in the final PCB design. These components are essential to boost the signal strength and integrity over the 2-meter cable length, countering the capacitive loading and ensuring reliable communication between the Brain and Sensor Nodes.

Regarding the actuator duty cycle, the control software will incorporate a hysteresis logic in the form of a "Time-Delay" filter. This filter will ensure the linear actuators are only commanded to move if a detected light differential persists for more than 60 seconds. This intentional delay prevents the system from reacting to transient shadows, such as those from passing clouds, thereby eliminating unnecessary short cycling. This protection is crucial to prevent motor overheating and reduce mechanical wear [11].

CHAPTER 5

FUTURE PLANS

5.1 MONOZUKURI 2 Goals

The transition from MONOZUKURI 1 (Proposal) to MONOZUKURI 2 (Implementation) will focus on physical fabrication:

- I. PCB Manufacturing: The schematic designs for the "Brain" and "Sensor Node" will be sent for professional fabrication to ensure reliability.
- II. Structural Fabrication: We will fabricate the custom steel brackets required to mount the linear actuators to standard aluminum solar frames.
- III. Field Testing: A beta unit will be deployed in an outdoor environment to validate the waterproofing (IP67) of the Spider Harness and gather real-world energy generation data to confirm the projected 40% efficiency gain.
- IV. Data Intelligence and Comprehensive Logging: To validate the system's long term efficiency, the next phase will focus on advanced data acquisition and automated reporting. We will implement a systematic "worksheet" approach where live telemetry from the Blynk IoT platform is automatically bridged to Google Sheets. This will enable high-resolution data logging in CSV format, allowing for the recording of detailed variables such as corner specific lux levels, real time wind forces, and horizontal/vertical motor positions. This data driven approach is essential for analyzing energy gains over time and refining the tracking algorithm based on seasonal variations.
- V. Physical Product Deployment and Real World Validation: Transitioning from a simulation based model to a physical product is the primary goal of the upcoming development cycle. This involves the fabrication of a full-scale prototype capable of reading actual solar output from physical PV panels rather than synthetic light signals. We will integrate power measurement modules (such as INA219 sensors) to directly quantify the wattage generated by the tracked system compared to a static baseline. This stage will also include the physical manufacturing of the custom steel mounting brackets and the linear actuator "Muscle" system to ensure mechanical stability under real world environmental loads.

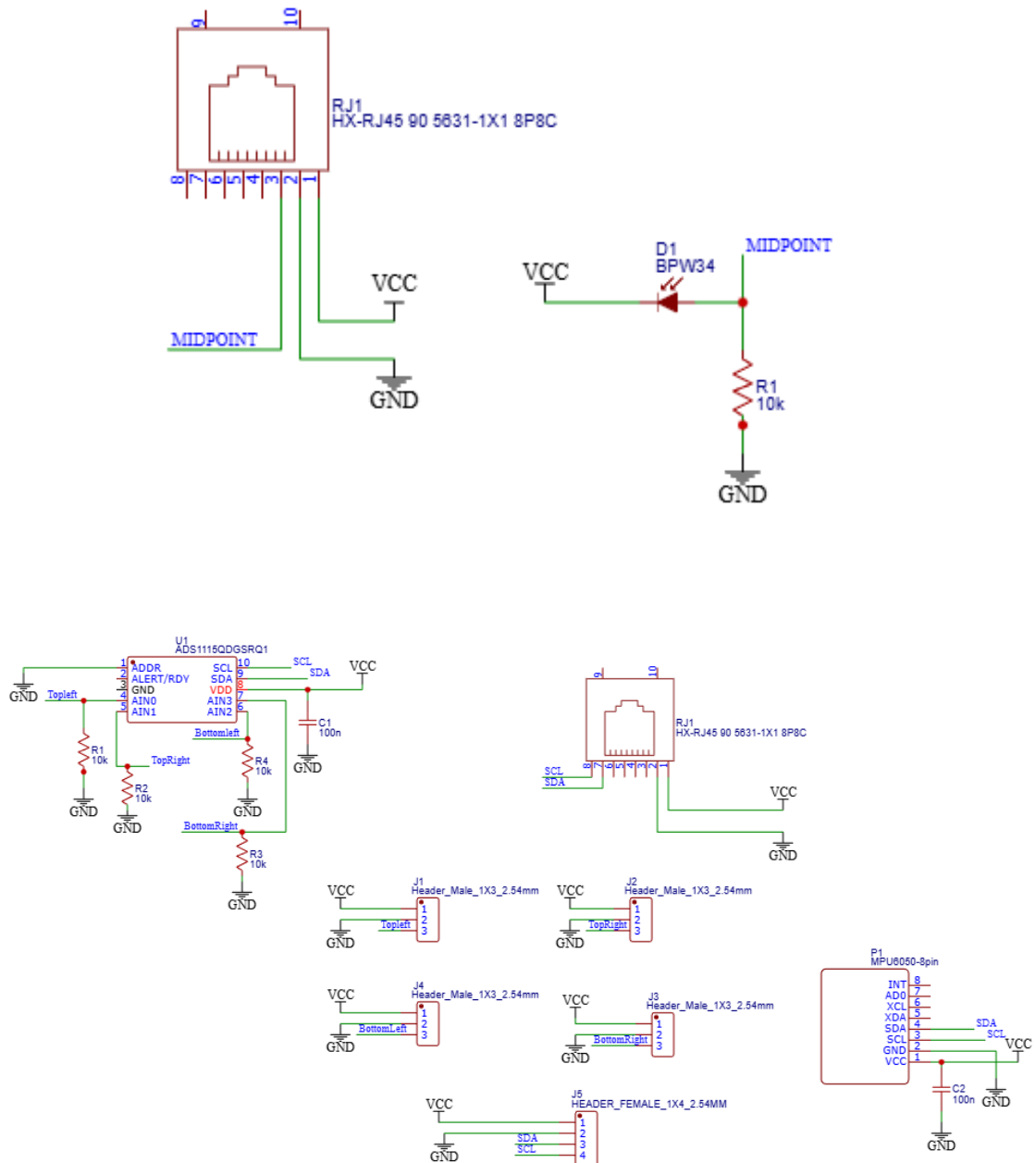
- VI. **Wireless Connectivity and Offline Accessibility:** Future iterations of the Sensol system will evolve from a wired sensor network to a fully wireless architecture to simplify installation further. By integrating wireless communication protocols like LoRa (Long Range), the system will gain "IoT Access" even in remote agricultural or off grid locations where Wi-Fi is unavailable. This transition eliminates the physical constraints of the 2-meter I2C cable, bypassing potential signal degradation issues while maintaining robust, long-distance data transmission for wide-scale solar farms.
- VII. **B2C and B2B Commercialization Strategy:**Our commercial vision includes a dual revenue model targeting both individual consumers and industrial partners. For the Business to Business (B2B) market, we will offer the "Sensol Pro Kit," which bundles the central controller hub, the "Spider Harness" sensor grid, and high torque actuators as a complete set and forget package. Complementing the hardware, we plan to launch a Software as a Service (SaaS) subscription model. This software will provide enterprise users with advanced predictive maintenance analytics, multi-array management dashboards, and automated efficiency reports, ensuring a sustained Return on Investment (ROI) over the product's 5 year lifecycle.

REFERENCES

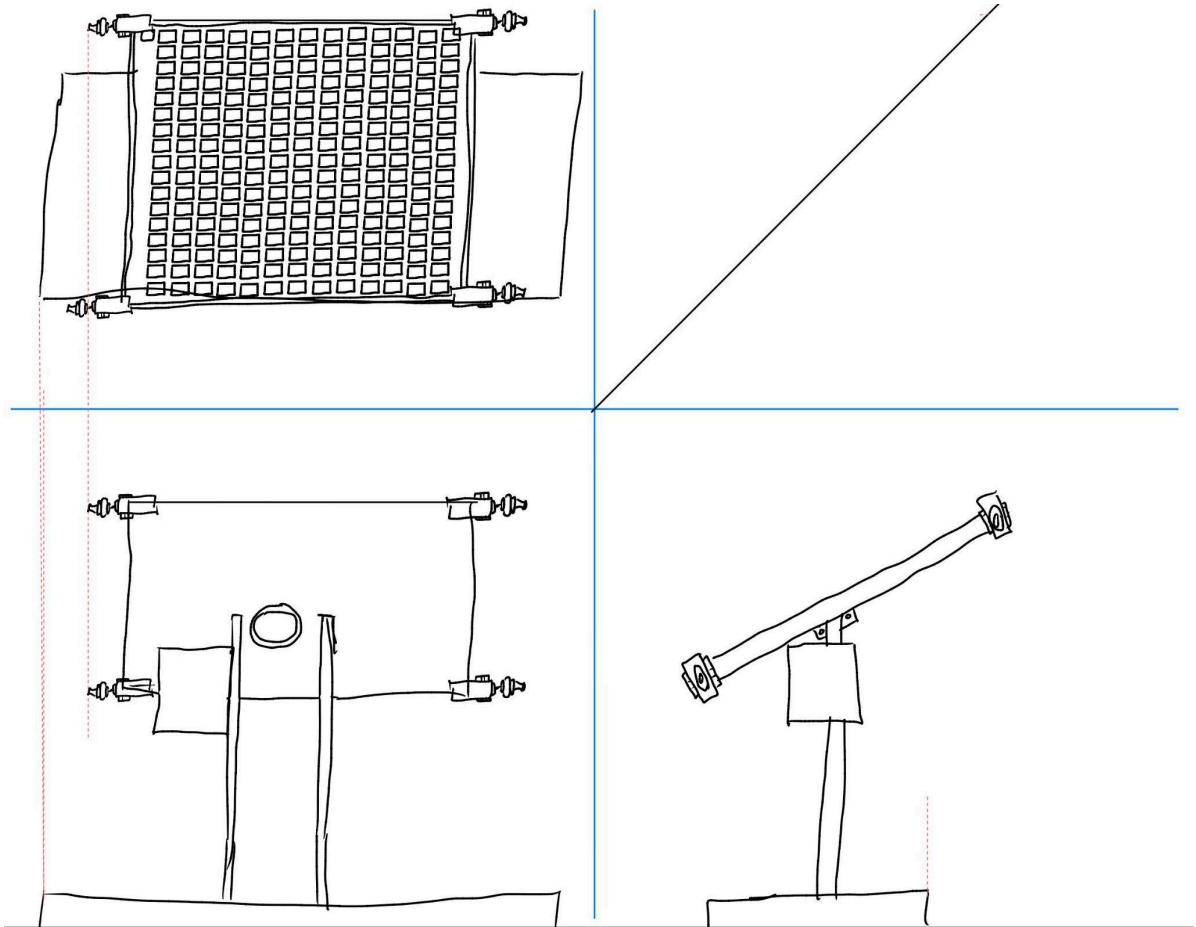
1. **TDK InvenSense.** (2013). *MPU-6000 and MPU-6050 Product Specification* (Rev. 3.4). TDK Corporation.
2. **Espressif Systems.** (2024). *ESP32 Series Datasheet* (Version 5.2). Espressif Systems.
3. **Bosch Sensortec.** (2016). *BME280: Combined Humidity and Pressure Sensor Datasheet* (Rev. 1.2). Bosch Sensortec.
4. **Blynk Inc.** (2024). *Blynk IoT Platform Documentation*. Retrieved from <https://docs.blynk.io>
5. **Meta AI.** (2024). *Llama 3 Model Card*. Retrieved from <https://github.com/meta-llama/llama3>
6. **Al-Sabounchi, A., Yousif, A., Kaddouri, W., & Ismail, S.** (2024). Solar tracking methods: A comprehensive survey. *International Journal for Research in Applied Science & Engineering Technology (IJRASET)*, 12(5), 1120–1127.
7. **Patel, A. P., & Dave, N. G.** (2025). A review and comparative analysis of solar tracking systems for enhanced photovoltaic energy capture. *Energies*, 18(12), 5890.
8. **Hamidin, M. F., & Sadun, A. S.** (2023). The development of IoT automatic solar tracking system for oriented solar cell applications. *Progress in Engineering Application and Technology*, 4(3), 78–85.
9. **Shukla, K. N., Rangnekar, S., & Sudhakar, K.** (2015). Solar tracking system: A review. *Renewable and Sustainable Energy Reviews*, 41, 675–684.
10. **Zulkafli, R. S., et al.** (2018). Dual axis solar tracking system in Perlis, Malaysia. *Journal of Telecommunication, Electronic and Computer Engineering*, 10(1–4), 87–92.
11. **Chong, K. K., & Wong, C. W.** (2009). General formula for on-axis sun-tracking system. *Solar Energy*, 83(2), 298–305.
12. **Abdallah, S., & Nijmeh, S.** (2004). Two-axis sun tracking system with PLC control. *Energy Conversion and Management*, 45(11–12), 1931–1939.
13. **Kumar, A., & Singh, B.** (2020). Design and implementation of LDR-based dual axis solar tracker. *International Journal of Engineering Research and Technology*, 9(6), 112–117.
14. **Flores-Hernández, D. A., Islas-Estrada, L. R., & Palomino-Resendiz, S. I.** (2024). A novel tracking strategy based on Real-Time monitoring to increase the lifetime of Dual-Axis solar tracking systems. *Applied Sciences*, 14(18), 8281.

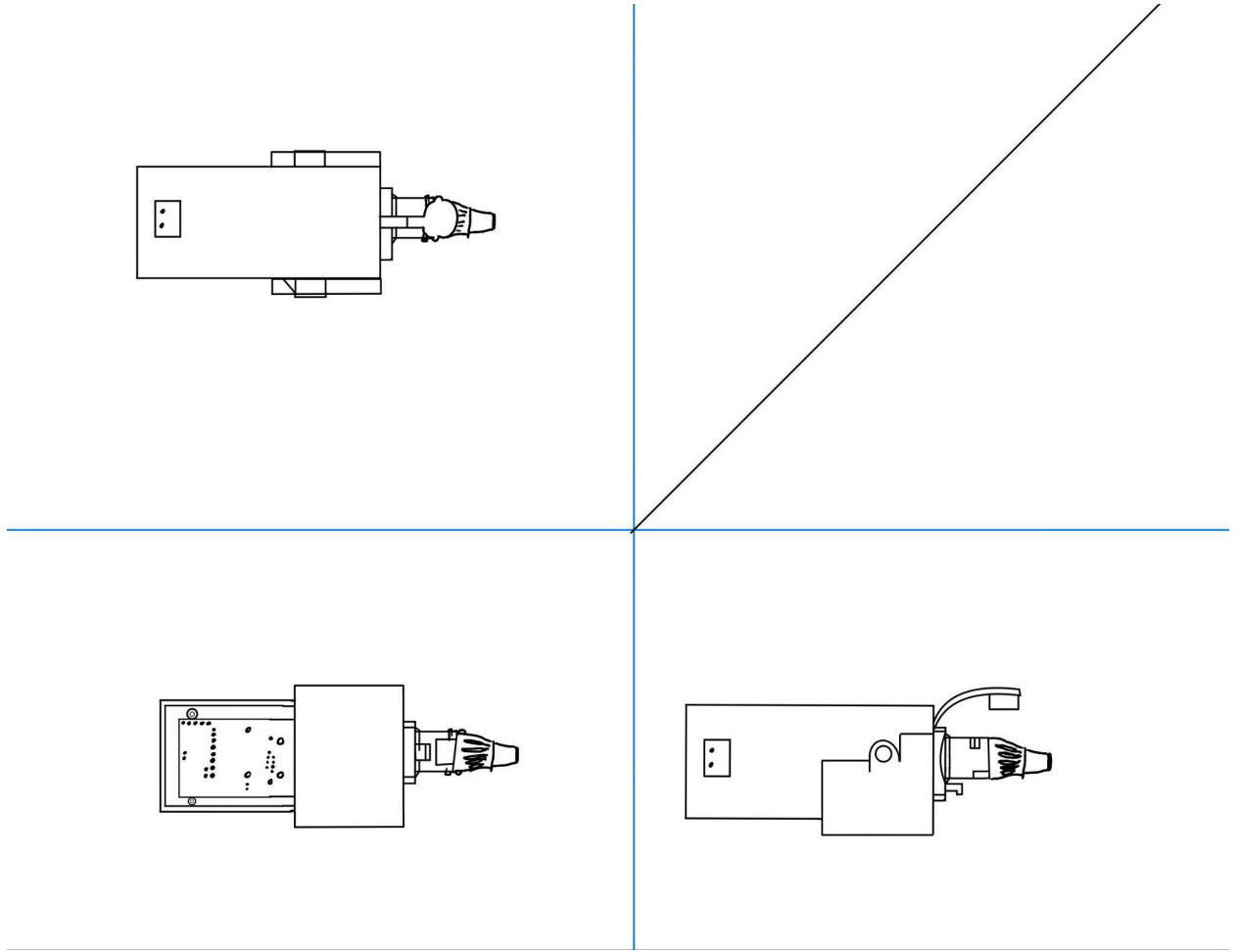
APPENDIX

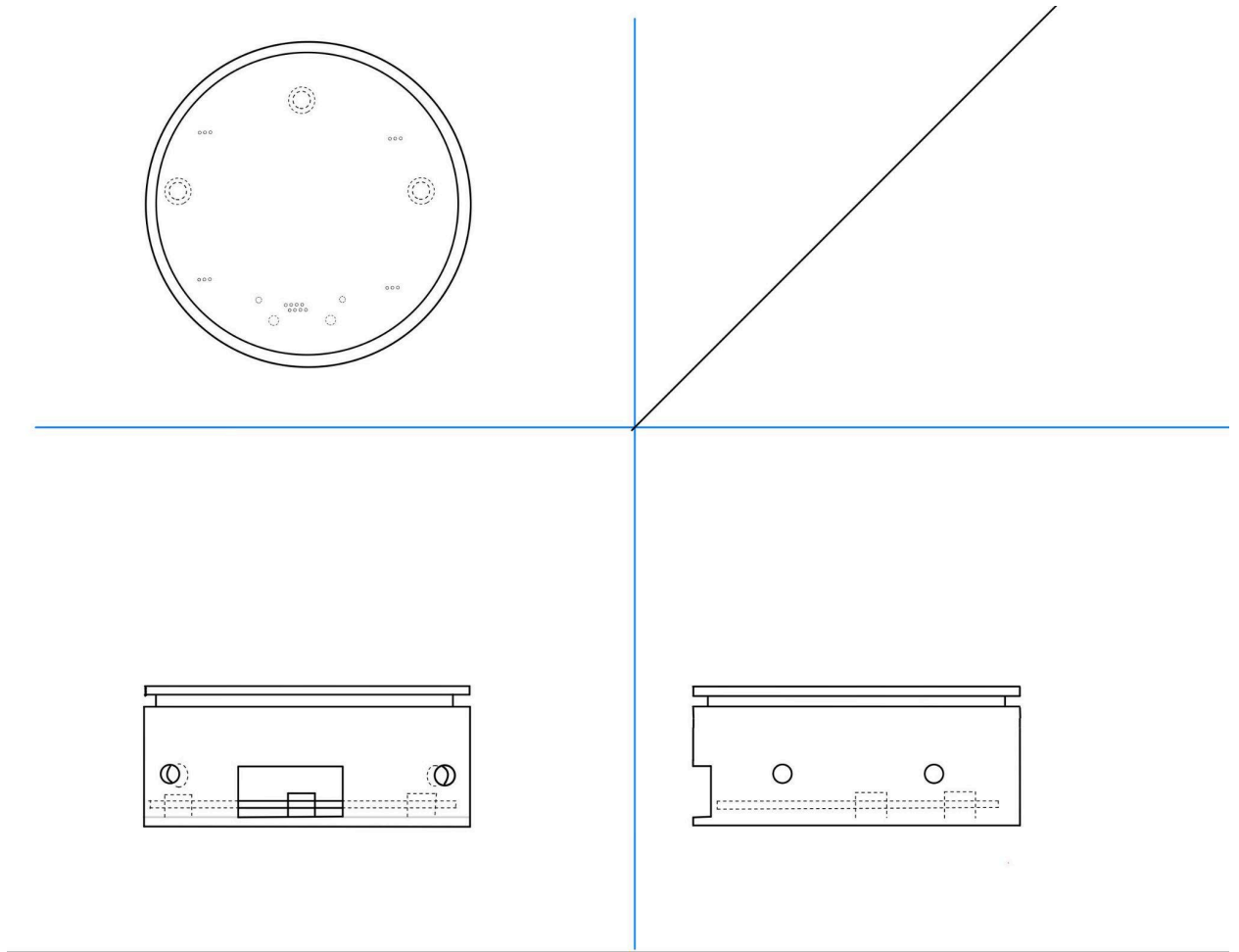
- Appendix A: COMPLETE CIRCUIT SCHEMATIC

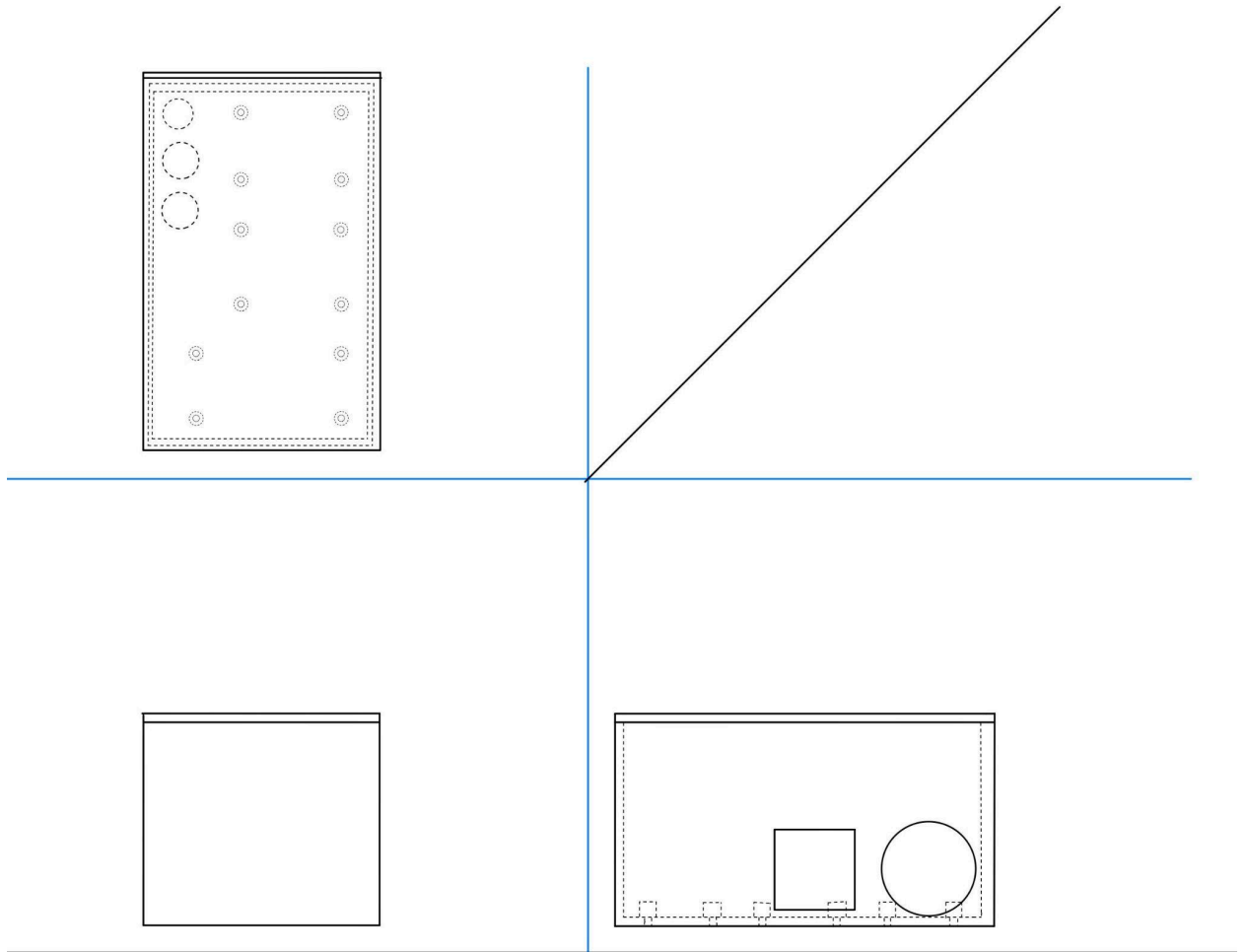


- Appendix B: 2D TECHNICAL DRAWING

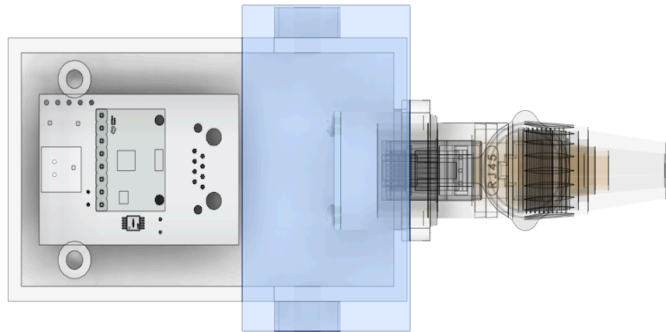
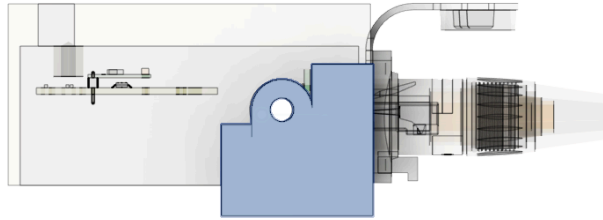


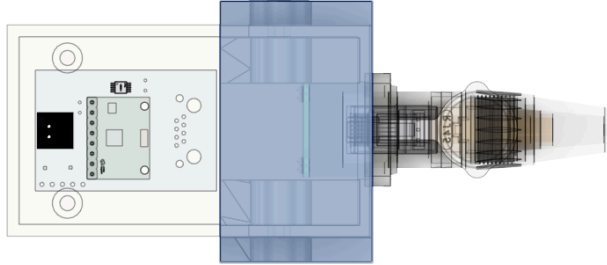


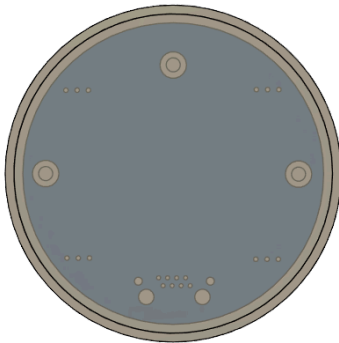
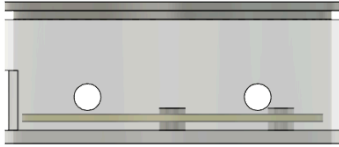




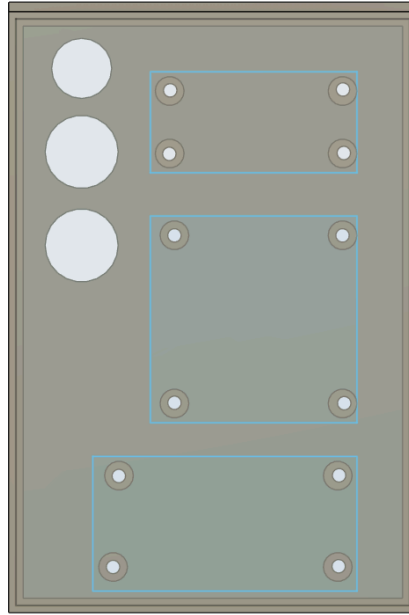
- Appendix C: TRANSPARENT 3D RENDER OF SENSOR NODES AND SPIDER HUB











- Appendix D: Firmware Source Code (C++)

```
#define BLYNK_TEMPLATE_ID "TMPL6DC1Q98kP"

#define BLYNK_TEMPLATE_NAME "Helios Tracker"

#define BLYNK_AUTH_TOKEN "zVTaCRf_aJ0rsnib_pmBPxfFUMOlGaym"

#define BLYNK_PRINT Serial

#include <WiFi.h>

#include <WiFiClientSecure.h>

#include <HTTPClient.h>

#include <ArduinoJson.h>

#include <BlynkSimpleEsp32.h>
```

```

#include <ESP32Servo.h>

#include <Wire.h>

#include <Adafruit_MPU6050.h>

#include <Adafruit_Sensor.h>

#include <RTCLib.h>

#include "GyverBME280.h"

// -----

// ⚠ PASTE YOUR GROQ API KEY HERE ⚠

// -----

const char* groq_api_key = "";

// --- WiFi ---

char ssid[] = "Wokwi-GUEST";

char pass[] =
"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";

// --- Objects ---

Servo verticalMotor;

Servo horizontalMotor;

```

```
Adafruit_MPU6050 mpu;

RTC_DS1307 rtc;

GyverBME280 bme;

BlynkTimer timer;


// --- Pins ---

const int ldrTopLeft = 35;

const int ldrTopRight = 32;

const int ldrBottomLeft = 33;

const int ldrBottomRight = 39;

const int ldrReferenceSun = 34;

const int servoV_Pin = 12;

const int servoH_Pin = 13;


// --- Thresholds ---

const int CLOUD_THRESHOLD = 2000;

const int DIRT_DIFFERENCE = 1000;

const float WIND_THRESHOLD = 1.5;
```

```

// --- Global Vars ---

int vPos = 90, hPos = 90;

int manualV = 90, manualH = 90;

String currentStateStr = "Booting...";

bool manualOverride = false;

float windForceGlobal = 0;

int averageLightGlobal = 0;

String timeStringGlobal = "00:00";


// =====

// PHYSICAL ANIMATION (Body Language)

// =====

void servoWiggle() {

    if (!manualOverride && windForceGlobal < WIND_THRESHOLD) {

        verticalMotor.write(vPos + 5);

        delay(100);

        verticalMotor.write(vPos - 5);

        delay(100);

        verticalMotor.write(vPos);
    }
}

```

```

    }

}

// =====

// GROQ AI (All-Knowing Context)

// =====

String askGroq(String userQuestion) {

    if (WiFi.status() != WL_CONNECTED) { WiFi.reconnect(); return
    "Error: WiFi Dropped"; }

    HTTPClient http;

    WiFiClientSecure client;

    client.setInsecure();

    http.setConnectTimeout(15000); http.setTimeout(15000);

    String url = "https://api.groq.com/openai/v1/chat/completions";

    if (!http.begin(client, url)) return "Error: HTTP Init Failed";

    http.addHeader("Content-Type", "application/json");

    http.addHeader("Authorization", "Bearer " + String(groq_api_key));

```



```

// 1. Gather Data

float hum = bme.readHumidity();

float temp = bme.readTemperature();

if (isnan(hum)) hum = 0; if (isnan(temp)) temp = 0;

String modeStr = manualOverride ? "MANUAL" : "AUTO";


// --- THE "ANALYST" PROMPT ---

String systemPrompt = "You are Helios, a smart solar tracker. ";


// A. THE DATA

systemPrompt += "Context: Time " + timeStringGlobal + ", Mode " +
modeStr + ". ";

systemPrompt += "Sensors: Wind " + String(windForceGlobal) + "G.
Light " + String(averageLightGlobal) + " lux. ";

systemPrompt += "Temp " + String(temp) + "C. Humidity " +
String(hum) + "%. ";


// B. THE BRAIN (How to analyze weather)

systemPrompt += "ANALYSIS RULES: ";

```

```

    systemPrompt += "1. High Wind (>1.5G) = Stormy. Recommendation:
STOW. ";

    systemPrompt += "2. Low Light (<1000) during Day = Cloudy/Rainy.
";

    systemPrompt += "3. High Light (>3000) = Sunny. Recommendation:
TRACK (Auto). ";

    systemPrompt += "4. High Humidity (>80%) + Low Light = Rain
likely. ";


// C. THE BEHAVIOR (Educated Guesses)

    systemPrompt += "INSTRUCTIONS: ";

    systemPrompt += "If the user asks about weather/status, analyze
the data provided above. ";

    systemPrompt += "Make an educated guess on the weather condition
(e.g., 'It seems like a storm is approaching...'). ";

    systemPrompt += "If the condition is dangerous or inefficient,
EXECUTE the adjustment yourself using commands. ";


// D. THE COMMANDS

    systemPrompt += "COMMANDS: [CMD:STOW], [CMD:AUTO], [CMD:WEST],
[CMD:EAST]. ";

    systemPrompt += "Keep answer conversational but under 30 words.";

```

```

JsonDocument doc;

doc["model"] = "llama-3.1-8b-instant";

JsonArray messages = doc.createNestedArray("messages");

JsonObject systemMessage = messages.createNestedObject();

systemMessage["role"] = "system"; systemMessage["content"] =
systemPrompt;

JsonObject userMessage = messages.createNestedObject();

userMessage["role"] = "user"; userMessage["content"] =
userQuestion;


String jsonPayload;

serializeJson(doc, jsonPayload);


int httpResponseCode = http.POST(jsonPayload);

String result = "Error";


if (httpResponseCode == 200) {

    String response = http.getString();

    JsonDocument responseDoc;

    deserializeJson(responseDoc, response);

```

```

    const char* answer =
responseDoc["choices"][0]["message"]["content"];

    if (answer) result = String(answer);

} else {

    result = "Error: " + String(httpResponseCode);

}

http.end();

return result;

}

// =====

// V5 BUTTON: TOGGLE MODE

// =====

BLYNK_WRITE(V5) {

    if (param.asInt() == 1) { // Only on Press

        manualOverride = !manualOverride; // Toggle

        if (manualOverride) {

            currentStateStr = "⚠ MANUAL: User Control";

```

```

    } else {

        currentStateStr = "☀️ Sunny. Tracking.";

        manualV = 90; manualH = 90;

        Blynk.virtualWrite(V10, 90);

        Blynk.virtualWrite(V11, 90);

    }

    Blynk.virtualWrite(V2, currentStateStr);

}

}

// =====

// CHAT AGENT (Agentic AI)

// =====

BLYNK_WRITE(V12) {

    String text = param.asStr();

    Blynk.virtualWrite(V12, "\n-----\n");

    Blynk.virtualWrite(V12, "YOU: " + text + "\n");

    Blynk.virtualWrite(V12, "Helios: Thinking...\n");

```

```

String aiReply = askGroq(text);

bool commandFound = false;

// --- 1. DETECT COMMANDS ---

// We check for the tag, execute the physical action, and flag it.

if (aiReply.indexOf("[CMD:STOW]") != -1) {

    manualOverride = true;

    manualV = 90; manualH = 90;

    vPos = 90; hPos = 90;

    Blynk.virtualWrite(V10, 90);

    Blynk.virtualWrite(V11, 90);

    currentStateStr = "⚠ MANUAL: AI Stowed Panel";

    Blynk.virtualWrite(V2, currentStateStr);

    commandFound = true;

}

```

```

if (aiReply.indexOf("[CMD:WEST]") != -1) {

    manualOverride = true;

    manualH = 180; manualV = 30;

    hPos = 180; vPos = 30;


    Blynk.virtualWrite(V11, 180);

    Blynk.virtualWrite(V10, 30);


    currentStateStr = "⚠ MANUAL: AI Facing West";

    Blynk.virtualWrite(V2, currentStateStr);

    commandFound = true;

}


if (aiReply.indexOf("[CMD:EAST]") != -1) {

    manualOverride = true;

    manualH = 0; manualV = 30;

    hPos = 0; vPos = 30;

```

```

    Blynk.virtualWrite(V11, 0);

    Blynk.virtualWrite(V10, 30);

    currentStateStr = "⚠ MANUAL: AI Facing East";

    Blynk.virtualWrite(V2, currentStateStr);

    commandFound = true;

}

if (aiReply.indexOf("[CMD:AUTO]") != -1) {

    manualOverride = false;

    manualV = 90; manualH = 90;

    currentStateStr = "☀ AI Resumed Auto-Tracking";

    Blynk.virtualWrite(V2, currentStateStr);

    commandFound = true;

}

// --- 2. CLEAN THE OUTPUT (The Fix) ---

// We simply replace the tags with an empty string ""

```



```

aiReply.replace("[CMD:STOW]", "");

aiReply.replace("[CMD:AUTO]", "");

aiReply.replace("[CMD:WEST]", "");

aiReply.replace("[CMD:EAST]", "");


// Also clean common variations just in case the AI messes up
slightly

aiReply.replace("[cmd:stow]", "");

aiReply.replace("[CMD: STOW]", "");


// Trim removes extra spaces left behind (e.g. "Text . " ->
"Text.")

aiReply.trim();


// --- 3. DISPLAY FINAL TEXT ---

if (!commandFound) servoWiggle();


Blynk.virtualWrite(V12, "Helios: " + aiReply + "\n");

Blynk.virtualWrite(V12, "-----\n");

```

```

}

// =====

// CONTROL LOGIC

// =====

BLYNK_WRITE(V10) { manualV = param.asInt(); }

BLYNK_WRITE(V11) { manualH = param.asInt(); }


void runSunnyTracking(int tl, int tr, int bl, int br) {

    int avgTop = (tl + tr) / 2;

    int avgBottom = (bl + br) / 2;

    int avgLeft = (tl + bl) / 2;

    int avgRight = (tr + br) / 2;

    int tolerance = 50;

    if (abs(avgTop - avgBottom) > tolerance) {

        if (avgTop > avgBottom) vPos++; else vPos--;

    }

    if (abs(avgLeft - avgRight) > tolerance) {

```

```

        if (avgLeft > avgRight) hPos++; else hPos--;

    }

    vPos = constrain(vPos, 10, 170);

    hPos = constrain(hPos, 0, 180);
}

void runSmartLogic() {

    DateTime now = rtc.now();

    int currentHour = now.hour();

    int currentMinute = now.minute();

    String suffix = (currentHour >= 12) ? " PM" : " AM";

    int hour12 = (currentHour > 12) ? currentHour - 12 :
currentHour;

    if (hour12 == 0) hour12 = 12;

    timeStringGlobal = String(hour12) + ":" + (currentMinute < 10 ?
"0" : "") + String(currentMinute) + suffix;

    int tl = analogRead(ldrTopLeft);

    int tr = analogRead(ldrTopRight);

```

```

int bl = analogRead(ldrBottomLeft);

int br = analogRead(ldrBottomRight);

int ref = analogRead(ldrReferenceSun);

averageLightGlobal = (tl + tr + bl + br) / 4;


sensors_event_t a, g, temp;

mpu.getEvent(&a, &g, &temp);

windForceGlobal = abs(a.acceleration.x) / 9.8;


float bmeTemp = bme.readTemperature();

float bmeHum = bme.readHumidity();

int alertLedStatus = 0;


static int loopCounter = 0;

loopCounter++;

bool sendData = (loopCounter >= 10);

if (sendData) loopCounter = 0;


if (manualOverride) {

```

```

        vPos = manualV; hPos = manualH;

    }

    else if (currentHour >= 19 || currentHour < 6) {

        currentStateStr = "🌙 Night Mode (" + timeStringGlobal +
")";

        vPos = 90; hPos = 0;

        alertLedStatus = 0;

    }

    else if (windForceGlobal > WIND_THRESHOLD) {

        currentStateStr = "⚠️ STORM: Panels Stowed";

        alertLedStatus = 1;

        vPos = 90; hPos = 90;

    }

    else if (ref > (averageLightGlobal + DIRT_DIFFERENCE)) {

        currentStateStr = "🧹 Dirt Alert! Tracking.";

        alertLedStatus = 1;

        runSunnyTracking(tl, tr, bl, br);

    }

    else if (averageLightGlobal < CLOUD_THRESHOLD) {

        currentStateStr = "☁️ Cloudy. Stowed";

```

```

        vPos = 90; hPos = 90;

    }

    else {

        currentStateStr = "☀ Sunny. Tracking.";

        runSunnyTracking(tl, tr, bl, br);

    }

    verticalMotor.write(vPos);

    horizontalMotor.write(hPos);

    if (sendData) {

        Blynk.virtualWrite(V0, averageLightGlobal);

        Blynk.virtualWrite(V1, windForceGlobal);

        Blynk.virtualWrite(V2, currentStateStr);

        Blynk.virtualWrite(V4, alertLedStatus);

        Blynk.virtualWrite(V6, vPos);

        Blynk.virtualWrite(V7, hPos);

        Blynk.virtualWrite(V8, bmeTemp);

        Blynk.virtualWrite(V9, bmeHum);

```

```

        Blynk.virtualWrite(V13, timeStringGlobal);

    }

}

void setup() {

    Serial.begin(115200);

    Wire.begin(21, 22);

    verticalMotor.attach(servoV_Pin);

    horizontalMotor.attach(servoH_Pin);

    if (!rtc.begin()) Serial.println("RTC Failed");

    else if (!rtc.isrunning()) rtc.adjust(DateTime(F(__DATE__),
F(__TIME__)));

    if (!mpu.begin(0x69)) Serial.println("MPU6050 Failed");

    else mpu.setAccelerometerRange(MPU6050_RANGE_8_G);

    if (!bme.begin(0x76)) {

        if (!bme.begin(0x77)) Serial.println("BME280 Failed");
    }
}

```

```
}

    Serial.print("Connecting to Blynk...");

    Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);

    Serial.println("Online");

    timer.setInterval(100L, runSmartLogic);
}

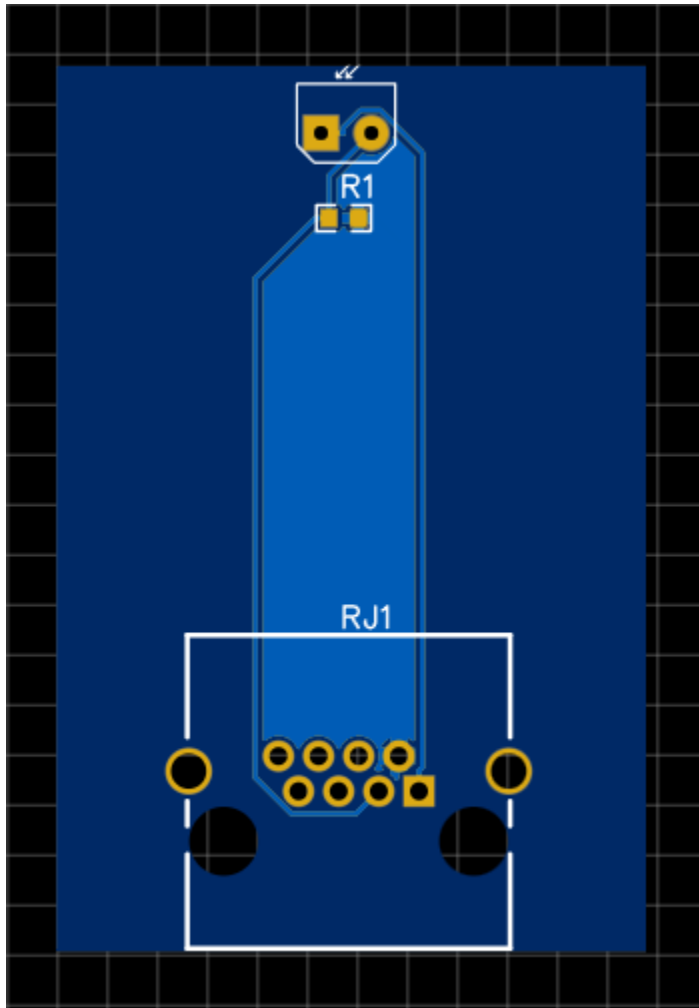
void loop() {

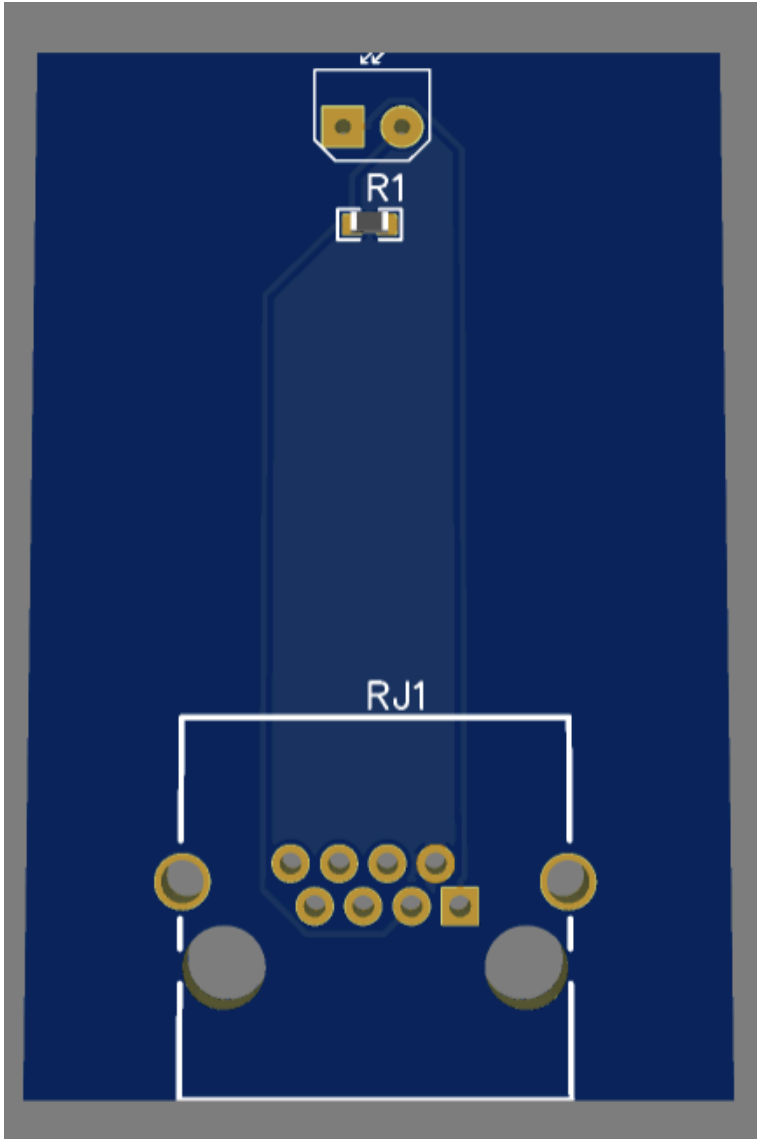
    Blynk.run();

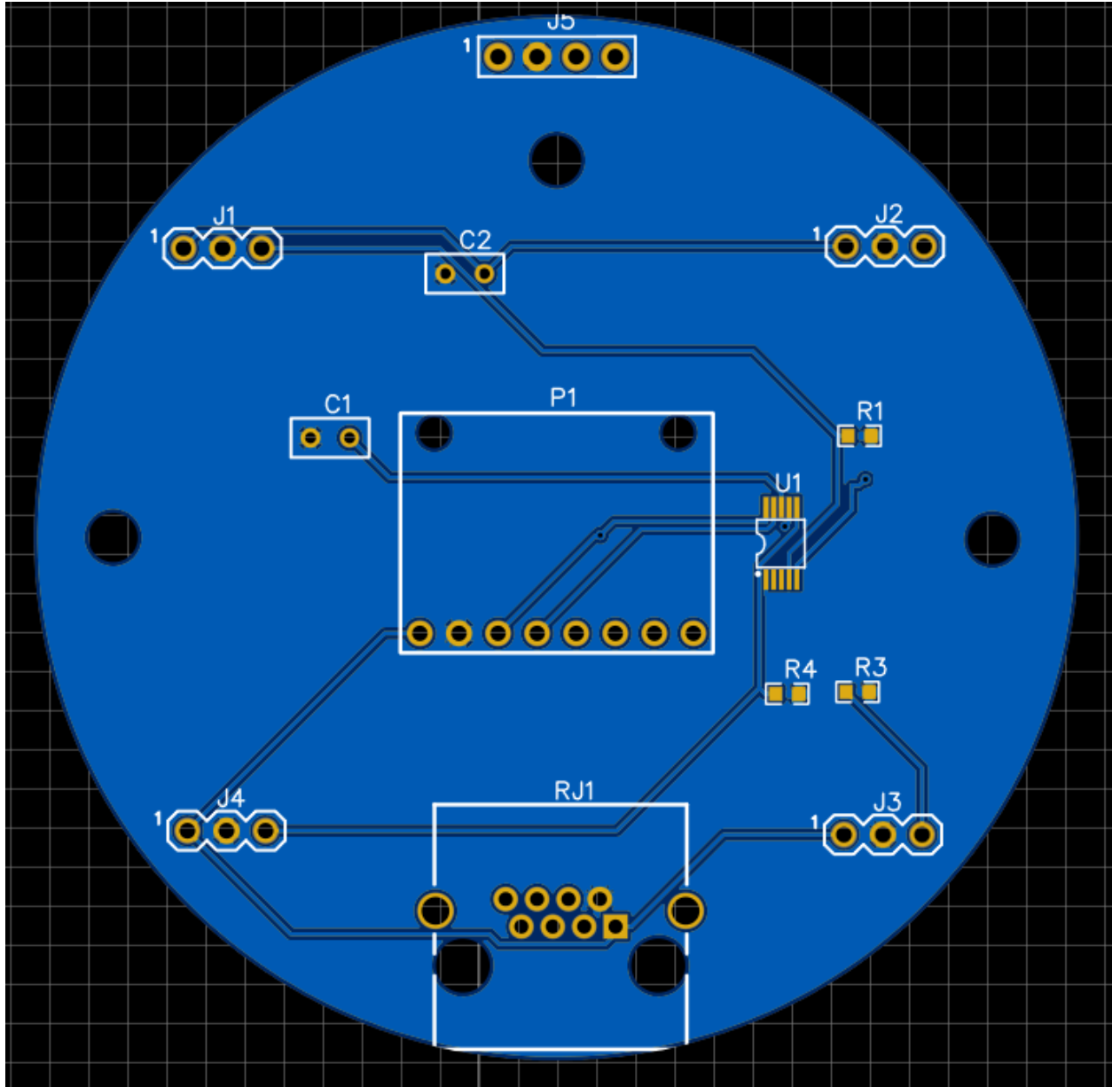
    timer.run();

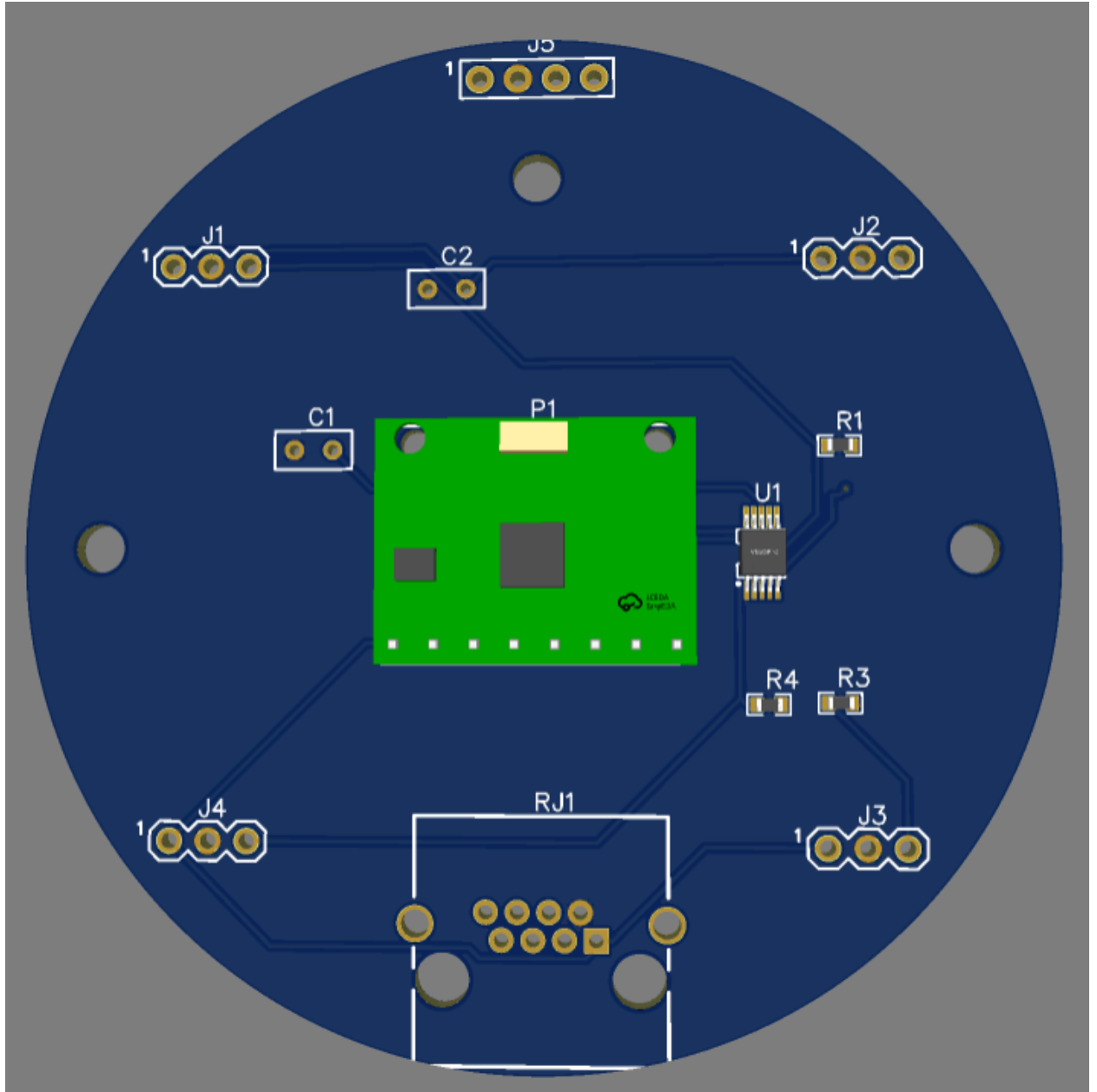
}
```


- Appendix E: 3D & 2D VIEW OF SENSOR NODE AND SPIDER HUB PCB









- Appendix F: Youtube Link Of Wokwi Simulation
 - <https://youtu.be/WPSBn1rdVBo>

UNIVERSITI TEKNOLOGI MALAYSIA
Malaysia-Japan International Institute of Technology (MJIT)
EVALUATION FORM
SMJE3813 (MONOZUKURI PROJECT)

#Name of students	Matrix No.	SCALE	Report 40%				
			0 - 5				
			(Introduction) (5%)	(Project Development (10%))	(Pitch Deck) (15%)	(Conclusion) (5%)	(Overall) (5%)
		POCO	CLO1, PLO1	CLO4, PLO4 (INVESTIGATE)	CLO3, PLO3 (DESIGN)	CLO5, PLO9 (COMMUNICATE)	CLO5, PLO9 (COMMUNICATE)
		Grad. Att.	SC	TH	TH	CS	CS
		Taxo	SC2	TH2	TH1	CS1	CS1
		Comp. Eng	WK1	WK8, WP2	WK5, WP1	EA2	EA2
		4~5	Clearly able to analyze the impact of the present work, in solving defined problems	The report very clearly explains, compares and summarizes the impact of existing work towards solving the problem.	The report showed that the project is well understood and correct methodologies have been identified and already applied to obtain some outcome	Comprehensive summary of the work are provided and future work included	Report very clear and sentences easy to understand. Minor or no grammatical errors. Report follows standard format.
		2~3	Fair and reasonably able to analyze the impact of the present work, in solving defined problems	The report sufficiently explains, compares and summarizes the impact of existing work towards solving the problem.	The report showed that the project is well understood but choice of problem solving approach is not entirely correct	Sufficient summary of the work are provided and future work included	Report well written but occasionally some points are not easy to understand. Some grammatical errors present. Standard format used.
		0~1	Minimal identification on the impact of the present work, in order to solve problems	The report poorly explains, explains, compares and summarizes the impact of existing work towards solving the problem.	The report shows that the project is not well understood or no evidence of any work being done.	Poorly summarize the project and no future work included	Report not well written with many grammatical errors or plagiarized work was presented.

1	ADIL AKID BIN ABD HAMID	.	A23MJ0021	.						
2	SIM HORNG VEI	.	A23MJ0111	.						
3	ARGYAN RIDHO AL GHIFARI	.	A23MJ8010	.						
4	IRFAN HAKIMI BIN ZULKIFLI	.	A23MJ0054	.						

Note: All Columns must be filled up